



中山大學  
SUN YAT-SEN UNIVERSITY

# 《计算机组成原理实验》 实验报告

(项目二)

学 院 名 称 : 计算机学院

专业 (班级) : 计算机科学与技术 9 班

学 生 姓 名 : 张玉瑶

学 号 : 23336316

时 间 : 2024 年 12 月 9 日

# 项目二：多周期CPU设计与实现

(套话可折叠)

## 一. 实验目的

- (1) 认识和掌握多周期数据通路图的构成、原理及其设计方法;
- (2) 掌握多周期 CPU 的实现方法, 代码实现方法;
- (3) 编写一个编译器, 将 MIPS 汇编程序编译为二进制机器码;
- (4) 掌握多周期 CPU 的测试方法;
- (5) 掌握多周期 CPU 的实现方法。

## 二. 实验内容

设计一个多周期 CPU, 该 CPU 在单周期指令集的基础上增加实现以下指令功能操作。需设计的指令与格式如下:

==>逻辑运算指令

(1) xori rt, rs, immediate

|        |         |         |                 |
|--------|---------|---------|-----------------|
| 001110 | rs(5 位) | rt(5 位) | immediate(16 位) |
|--------|---------|---------|-----------------|

功能:  $GPR[rt] \leftarrow GPR[rs] \text{ XOR } \text{zero\_extend}(\text{immediate})$ 。

==>比较指令

(1) slt rd, rs, rt 带符号

|        |         |         |         |              |
|--------|---------|---------|---------|--------------|
| 000000 | rs(5 位) | rt(5 位) | rd(5 位) | 00000 101010 |
|--------|---------|---------|---------|--------------|

功能: if ( $GPR[rs] < GPR[rt]$ )  $GPR[rd] = 1$  else ( $GPR[rd] = 0$ 。

==>跳转指令

(1) jr rs

|        |         |    |    |        |
|--------|---------|----|----|--------|
| 000000 | rs(5 位) | 未用 | 未用 | 001000 |
|--------|---------|----|----|--------|

功能:  $PC \leftarrow GPR[rs]$ , 跳转。

==>调用子程序指令

(2) jal addr

|        |            |
|--------|------------|
| 000011 | addr[27:2] |
|--------|------------|

功能: 调用子程序,  $PC \leftarrow \{PC[31:28], \text{addr}, 2'b0\}$ ;  $GPR[\$31] \leftarrow pc+4$ , 返回地址设置; 子程序返回, 需用指令 jr \$31。跳转地址的形成同 j addr 指令。

## 三. 实验原理

多周期 CPU 指的是将整个 CPU 的执行过程分成几个阶段, 每个阶段用一个时钟去完成, 然后开始下一条指令的执行, 而每种指令执行时所用的时钟数不尽相

同，这就是所谓的多周期 CPU。CPU 在处理指令时，一般需要经过以下几个阶段：

(1) 取指令(IF)：根据程序计数器 pc 中的指令地址，从存储器中取出一条指令，同时，pc 根据指令字长度自动递增产生下一条指令所需要的指令地址，但遇到“地址转移”指令时，则控制器把“转移地址”送入 pc，当然得到的“地址”需要做些变换才送入 pc。

(2) 指令译码(ID)：对取指令操作中得到的指令进行分析并译码，确定这条指令需要完成的操作，从而产生相应的操作控制信号，用于驱动执行状态中的各种操作。

(3) 指令执行(EXE)：根据指令译码得到的操作控制信号，具体地执行指令动作，然后转移到结果写回状态。

(4) 存储器访问(MEM)：所有需要访问存储器的操作都将在这个步骤中执行，该步骤给出存储器的数据地址，把数据写入到存储器中数据地址所指定的存储单元或者从存储器中得到数据地址单元中的数据。

(5) 结果写回(WB)：指令执行的结果或者访问存储器中得到的数据写回相应的目的寄存器中。

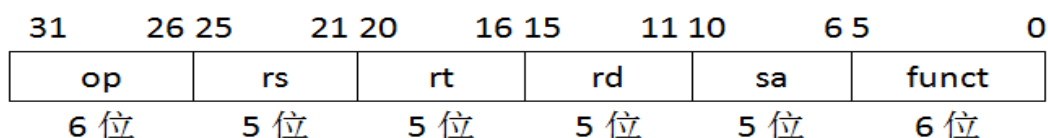
实验中就按照这五个阶段进行设计，这样一条指令的执行最长需要五个(小)时钟周期才能完成，但具体情况怎样？要根据该条指令的情况而定，有些指令不需要五个时钟周期的，这就是多周期的 CPU。



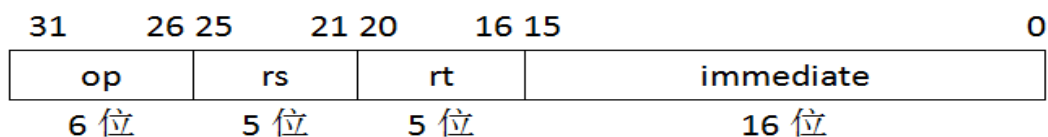
图 1 多周期 CPU 指令处理过程

MIPS 指令的三种格式：

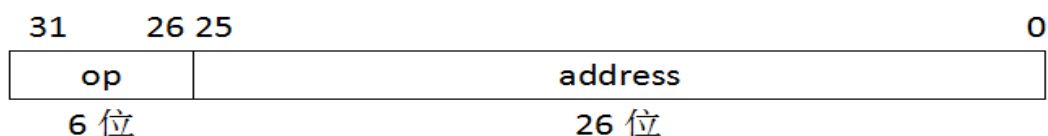
**R 类型：**



**I 类型：**



**J 类型：**



其中，

op：为操作码；

rs：为第 1 个源操作数寄存器，寄存器地址(编号)是 00000~11111, 00~1F；

rt：为第 2 个源操作数寄存器，或目的操作数寄存器，寄存器地址(同上)；

**rd:** 为目的操作数寄存器，寄存器地址（同上）；  
**sa:** 为位移量（shift amt），移位指令用于指定移多少位；  
**funct:** 为功能码，在寄存器类型指令中（R 类型）用来指定指令的功能；  
**immediate:** 为 16 位立即数，用作无符号的逻辑操作数、有符号的算术操作数、数据加载（Load）/数据保存（Store）指令的数据地址字节偏移量和分支指令中相对程序计数器（PC）的有符号偏移量；  
**address:** 为地址。

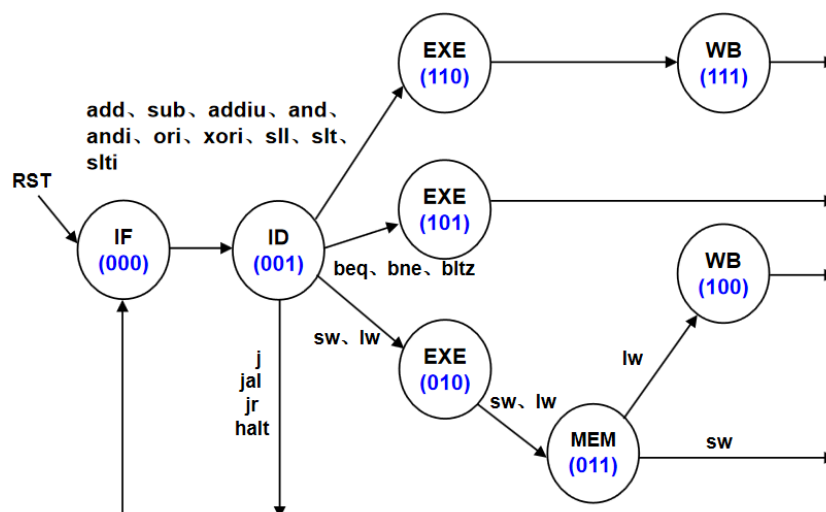


图 2 多周期 CPU 状态转移图

状态的转移有的是无条件的，例如从 sIF 状态转移到 sID 就是无条件的；有些是有条件的，例如 sEXE 状态之后不止一个状态，到底转向哪个状态由该指令功能，即指令操作码决定。每个状态代表一个时钟周期。

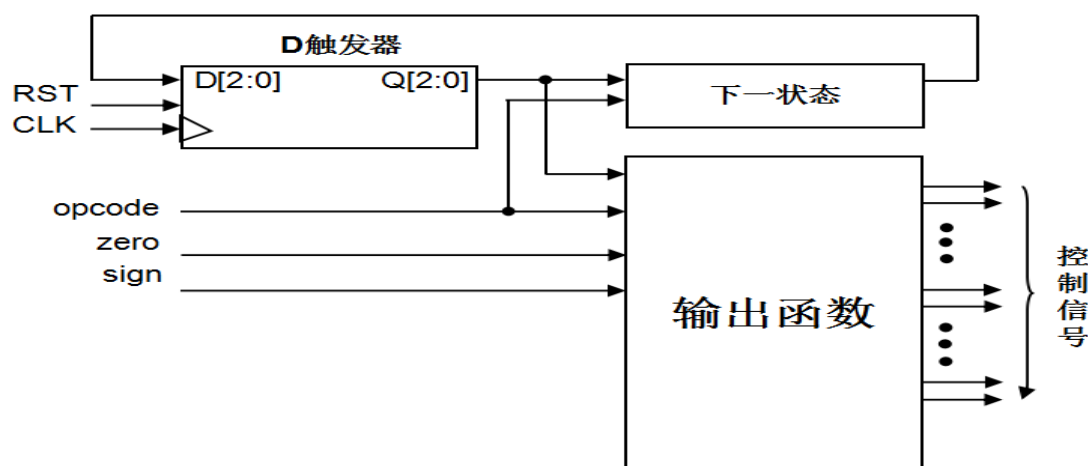


图 3 多周期 CPU 控制部件的原理结构图

图 3 是多周期 CPU 控制部件的电路结构，三个 D 触发器用于保存当前状态，是时序逻辑电路，RST 用于初始化状态“000”，另外两个部分都是组合逻辑电路，一个用于产生下一个阶段的状态，另一个用于产生每个阶段的控制信号。从图上可看出，下个状态取决于指令操作码和当前状态；而每个阶段的控制信号取决于指令操作码、当前状态和反映运算结果的状态 zero 标志和符号 sign 标志。

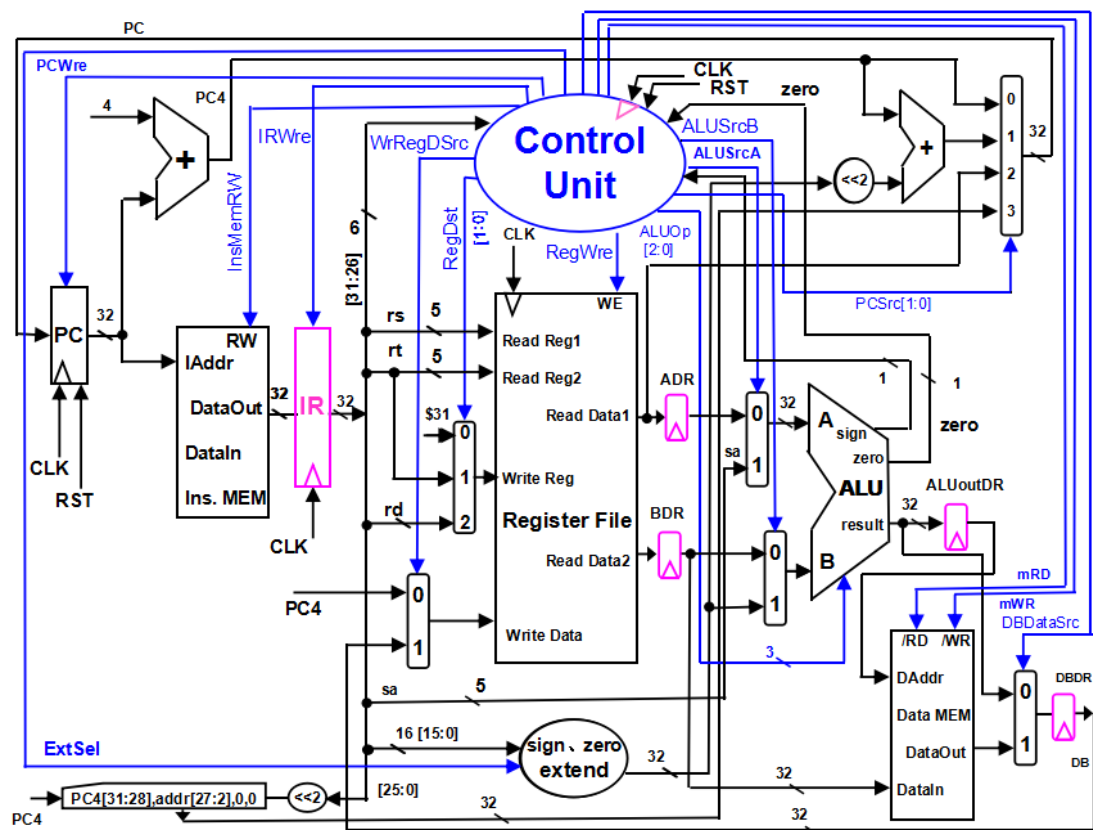


图 4 多周期 CPU 数据通路和控制线路图

图 4 是一个简单的基本上能够在多周期 CPU 上完成所要求设计的指令功能的数据通路和必要的控制线路图。其中指令和数据各存储在不同存储器中，即有指令存储器和数据存储器。访问存储器时，先给出内存地址，然后由读或写信号控制操作。对于寄存器组，给出寄存器地址（编号），读操作时不需要时钟信号，输出端就直接输出相应数据；而在写操作时，在 WE 使能信号为 1 时，在时钟边沿触发将数据写入寄存器。图中控制信号功能如表 1 所示，表 2 是 ALU 运算功能表。

特别提示，图上增加 IR 指令寄存器，目的是使指令代码保持稳定，pc 写使能控制信号 PCWre，是确保 pc 适时修改，原因都是和多周期工作的 CPU 有关。ADR、BDR、ALUoutDR、DBDR 四个寄存器不需要写使能信号，其作用是切分数据通路，将大组合逻辑切分为若干个小组合逻辑，大延迟变为多个分段小延迟。

表 1 控制信号作用

| 控制信号名      | 状态“0”                                                 | 状态“1”                                                            |
|------------|-------------------------------------------------------|------------------------------------------------------------------|
| <b>RST</b> | 对于 PC，初始化 PC 为程序首地址                                   | 对于 PC，PC 接收下一条指令地址                                               |
| PCWre      | PC 不更改，相关指令：halt，另外，除‘000’状态之外，其余状态慎改 PC 的值。          | PC 更改，相关指令：除指令 halt 外，另外，在‘000’状态时，修改 PC 的值合适。                   |
| ALUSrcA    | 来自寄存器堆 data1 输出，相关指令：add、sub、addiu、and、andi、ori、xori、 | 来自移位数 sa，同时，进行 (zero-extend) sa，即 $\{27\{1'b0\}, sa\}$ ，相关指令：sll |

|              |                                                                                                                                                                                                                                                                                                                                  |                                                                                       |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
|              | slt、slti、sw、lw、beq、bne、bltz                                                                                                                                                                                                                                                                                                      |                                                                                       |
| ALUSrcB      | 来自寄存器堆 data2 输出，相关指令：add、sub、and、slt、sll、beq、bne、bltz                                                                                                                                                                                                                                                                            | 来自 sign 或 zero 扩展的立即数，相关指令：addiu、andi、ori、xori、slti、lw、sw                             |
| DBDataSrc    | 来自 ALU 运算结果的输出，相关指令：add、sub、addiu、and、andi、ori、xori、sll、slt、slti                                                                                                                                                                                                                                                                 | 来自数据存储器（Data MEM）的输出，相关指令：lw                                                          |
| RegWre       | 无写寄存器组寄存器，相关指令：beq、bne、bltz、j、sw、jr、halt                                                                                                                                                                                                                                                                                         | 寄存器组寄存器写使能，相关指令：add、sub、addiu、and、andi、ori、xori、sll、slt、slti、lw、jal                   |
| WrRegDSrc    | 写入寄存器组寄存器的数据来自 pc+4(pc4)，相关指令：jal，写\$31                                                                                                                                                                                                                                                                                          | 写入寄存器组寄存器的数据来自 ALU 运算结果或存储器读出的数据，相关指令：add、addiu、sub、and、andi、ori、xori、sll、slt、slti、lw |
| InsMemRW     | 写指令存储器                                                                                                                                                                                                                                                                                                                           | 读指令存储器(Ins. Data)                                                                     |
| mRD          | 存储器输出高阻态                                                                                                                                                                                                                                                                                                                         | 读数据存储器，相关指令：lw                                                                        |
| mWR          | 无操作                                                                                                                                                                                                                                                                                                                              | 写数据存储器，相关指令：sw                                                                        |
| IRWre        | IR(指令寄存器)不更改                                                                                                                                                                                                                                                                                                                     | IR 寄存器写使能。向指令存储器发出读指令代码后，这个信号也接着发出，在时钟上升沿，IR 接收从指令存储器送来的指令代码。与每条指令都相关。                |
| ExtSel       | (zero-extend) <b>immediate</b> ，相关指令：andi、xori、ori；                                                                                                                                                                                                                                                                              | (sign-extend) <b>immediate</b> ，相关指令：addiu、slti、lw、sw、beq、bne、bltz；                   |
|              |                                                                                                                                                                                                                                                                                                                                  |                                                                                       |
| PCSrc[1..0]  | 00: $pc < -pc+4$ ，相关指令：add、addiu、sub、and、andi、ori、xori、slt、slti、sll、sw、lw、beq(zero=0)、bne(zero=1)、bltz(sign=0)；<br>01: $pc < -pc+4 + (\text{sign-extend}) \text{immediate} \times 4$ ，相关指令：beq(zero=1)、bne(zero=0)、bltz(sign=1)；<br>10: $pc < -rs$ ，相关指令：jr；<br>11: $pc < -\{pc[31:28], \text{addr}[27:2], 2'b00\}$ ，相关指令：j、jal； |                                                                                       |
| RegDst[1..0] | 写寄存器组寄存器的地址，来自：<br>00: 0x1F(\$31)，相关指令：jal，用于保存返回地址（\$31<-pc+4）；<br>01: rt 字段，相关指令：addiu、andi、ori、xori、slti、lw；                                                                                                                                                                                                                  |                                                                                       |

|             |                                                  |
|-------------|--------------------------------------------------|
|             | 10: rd 字段, 相关指令: add、sub、and、slt、sll;<br>11: 未用; |
| ALUOp[2..0] | ALU 8 种运算功能选择(000-111), 看功能表                     |

相关部件及引脚说明:

**Instruction Memory: 指令存储器**

- Iaddr, 指令地址输入端口
- DataIn, 存储器数据输入端口
- DataOut, 存储器数据输出端口
- RW, 指令存储器读写控制信号, 为 0 写, 为 1 读

**Data Memory: 数据存储器**

- Daddr, 数据地址输入端口
- DataIn, 存储器数据输入端口
- DataOut, 存储器数据输出端口
- /RD, 数据存储器读控制信号, 为 0 读
- /WR, 数据存储器写控制信号, 为 0 写

**Register File: 寄存器组**

- Read Reg1, rs 寄存器地址输入端口
- Read Reg2, rt 寄存器地址输入端口
- Write Reg, 将数据写入的寄存器, 其地址输入端口 (rt、rd)
- Write Data, 写入寄存器的数据输入端口
- Read Data1, rs 寄存器数据输出端口
- Read Data2, rt 寄存器数据输出端口
- WE, 写使能信号, 为 1 时, 在时钟边沿触发写入

**IR: 指令寄存器**, 用于存放正在执行的指令代码

**ALU: 算术逻辑单元**

- result, ALU 运算结果
- zero, 运算结果标志, 结果为 0, 则 zero=1; 否则 zero=0
- sign, 运算结果标志, 结果最高位为 0, 则 sign=0, 正数; 否则, sign=1, 负数

表 2 ALU 运算功能表

| ALUOp[2..0] | 功能                                                                                  | 描述             |
|-------------|-------------------------------------------------------------------------------------|----------------|
| 000         | $Y = A + B$                                                                         | 加              |
| 001         | $Y = A - B$                                                                         | 减              |
| 010         | $Y = B \ll A$                                                                       | B 左移 A 位       |
| 011         | $Y = A \vee B$                                                                      | 或              |
| 100         | $Y = A \wedge B$                                                                    | 与              |
| 101         | $Y = (A < B) ? 1 : 0$                                                               | 比较 A<B<br>不带符号 |
| 110         | $Y = (((A < B) \&\& (A[31] == B[31])) \vee ((A[31] == 1 \&\& B[31] == 0))) ? 1 : 0$ | 比较 A<B<br>带符号  |

|     |                  |    |
|-----|------------------|----|
| 111 | $Y = A \oplus B$ | 异或 |
|-----|------------------|----|

## 四．实验设备

PC 机一台，BASYS 3 实验板一块，Xilinx Vivado 开发软件一套。

## 五．实验内容和结果

多周期 CPU 在单周期 CPU 的基础上增多了若干寄存器，并在控制器上区分了不同状态下的控制信号。以下是各个模块的设计：

### 1. 控制器

每条指令分为若干阶段，并在每个阶段对信号做出了区分。

|    | A          | B     | C      | D      | E   | F     | G         | H      | I       | J       | K         | L        | M   | N   | O     | P      | Q      | R          | S     |
|----|------------|-------|--------|--------|-----|-------|-----------|--------|---------|---------|-----------|----------|-----|-----|-------|--------|--------|------------|-------|
|    | 状态         | 指令    | op码    | func码  | RST | PCWre | DBDataSrc | RegWre | ALUSrcA | ALUSrcB | WrRegDsrc | InsMemRW | mRD | mWR | IRWre | ExtSel | RegDst | PCSrc      | ALUOP |
| 1  | 000(取指)    | 所有    |        |        | 1   | 1     | x         | 0      | x       | x       | x         | 1        | 1   | 0   | 1     | x      | x      | x          | xxx   |
| 2  |            | j     | 000010 |        | 1   | 0     | x         | 0      | x       | x       | x         | 1        | 1   | 0   | 0     | x      | xx     | 11         | xxx   |
| 3  |            | jal   | 000011 |        | 1   | 0     | x         | 1      | x       | x       | 0         | 1        | 1   | 0   | 0     | x      | 00     | 11         | xxx   |
| 4  |            | jr    | 000000 | 001000 | 1   | 0     | x         | 0      | x       | x       | x         | 1        | 1   | 0   | 0     | x      | xx     | 10         | xxx   |
| 5  | 001(译码)    | halt  | 111111 |        | 1   | 0     | x         | 0      | x       | x       | x         | 1        | 1   | 0   | 0     | x      | xx     | x          | xxx   |
| 6  |            | sw    | 101011 |        | 1   | 0     | x         | 0      | 0       | 1       | x         | 1        | 1   | 0   | 0     | 1      | xx     | xx         | 000   |
| 7  | 010(地址计算)  | lw    | 100011 |        | 1   | 0     | x         | 0      | 0       | 1       | x         | 1        | 1   | 0   | 0     | 1      | xx     | xx         | 000   |
| 8  |            | sw    | 101011 |        | 1   | 0     | x         | 0      | x       | x       | x         | 1        | 1   | 1   | 0     | x      | xx     | 00         | xxx   |
| 9  | 011(访存)    | lw    | 100011 |        | 1   | 0     | x         | 0      | x       | x       | x         | 1        | 1   | 0   | 0     | x      | xx     | xx         | xxx   |
| 10 | 100(写回)    | lw    | 100011 |        | 1   | 0     | 1         | 1      | x       | x       | 1         | 1        | 1   | 0   | 0     | x      | 01     | 00         | xxx   |
| 11 |            | beq   | 000100 |        | 1   | 0     | x         | 0      | 0       | 0       | x         | 1        | 1   | 0   | 0     | 1      | xx     | zero==1701 | 001   |
| 12 |            | bne   | 000101 |        | 1   | 0     | x         | 0      | 0       | 0       | x         | 1        | 1   | 0   | 0     | 1      | xx     | zero==1701 | 001   |
| 13 | 101(条件判断)  | blez  | 000110 |        | 1   | 0     | x         | 0      | 0       | 0       | x         | 1        | 1   | 0   | 0     | 1      | xx     | zero==1701 | 001   |
| 14 |            | add   | 000000 | 100000 | 1   | 0     | x         | 0      | 0       | 0       | x         | 1        | 1   | 0   | 0     | 0      | 10     | xx         | 000   |
| 15 |            | sub   | 000000 | 100010 | 1   | 0     | x         | 0      | 0       | 0       | x         | 1        | 1   | 0   | 0     | 0      | 10     | xx         | 001   |
| 16 |            | addiu | 001001 |        | 1   | 0     | x         | 0      | 0       | 1       | x         | 1        | 1   | 0   | 0     | 1      | 10     | xx         | 000   |
| 17 |            | and   | 000000 | 100100 | 1   | 0     | x         | 0      | 0       | 0       | x         | 1        | 1   | 0   | 0     | 0      | 10     | xx         | 100   |
| 18 |            | andi  | 001100 |        | 1   | 0     | x         | 0      | 0       | 1       | x         | 1        | 1   | 0   | 0     | 0      | 10     | xx         | 100   |
| 19 |            | ori   | 001101 |        | 1   | 0     | x         | 0      | 0       | 1       | x         | 1        | 1   | 0   | 0     | 0      | 10     | xx         | 011   |
| 20 |            | xori  | 001110 |        | 1   | 0     | x         | 0      | 0       | 1       | x         | 1        | 1   | 0   | 0     | 0      | 10     | xx         | 011   |
| 21 |            | sll   | 000000 | 000000 | 1   | 0     | x         | 0      | 1       | 0       | x         | 1        | 1   | 0   | 0     | 0      | 10     | xx         | 010   |
| 22 |            | slt   | 000000 | 101010 | 1   | 0     | x         | 0      | 0       | 0       | x         | 1        | 1   | 0   | 0     | 0      | 10     | xx         | 101   |
| 23 |            | or    | 000000 | 100101 | 1   | 0     | x         | 0      | 0       | 0       | x         | 1        | 1   | 0   | 0     | 0      | 10     | xx         | 011   |
| 24 | 110(ALU计算) | sli   | 001010 |        | 1   | 0     | x         | 0      | 0       | 1       | x         | x        | x   | 0   | 0     | 1      | 10     | xx         | 110   |
| 25 |            | add   | 000000 | 100000 | 1   | 0     | 0         | 1      | x       | x       | 1         | 1        | 1   | 0   | 0     | 0      | 10     | 00         | 000   |
| 26 |            | sub   | 000000 | 100010 | 1   | 0     | 0         | 1      | x       | x       | 1         | 1        | 1   | 0   | 0     | 0      | 10     | 00         | 001   |
| 27 |            | addiu | 001001 |        | 1   | 0     | 0         | 1      | x       | x       | 1         | 1        | 1   | 0   | 0     | 0      | 01     | 00         | 000   |
| 28 |            | and   | 000000 | 100100 | 1   | 0     | 0         | 1      | x       | x       | 1         | 1        | 1   | 0   | 0     | 0      | 10     | 00         | 100   |
| 29 |            | andi  | 001100 |        | 1   | 0     | 0         | 1      | x       | x       | 1         | 1        | 1   | 0   | 0     | 0      | 01     | 00         | 100   |
| 30 |            | ori   | 001101 |        | 1   | 0     | 0         | 1      | x       | x       | 1         | 1        | 1   | 0   | 0     | 0      | 01     | 00         | 011   |
| 31 |            | xori  | 001110 |        | 1   | 0     | 0         | 1      | x       | x       | 1         | 1        | 1   | 0   | 0     | 0      | 01     | 00         | 011   |
| 32 |            | sll   | 000000 | 000000 | 1   | 0     | 0         | 1      | x       | x       | 1         | 1        | 1   | 0   | 0     | 0      | 10     | 00         | 010   |
| 33 |            | slt   | 000000 | 101010 | 1   | 0     | 0         | 1      | x       | x       | 1         | 1        | 1   | 0   | 0     | 0      | 10     | 00         | 101   |
| 34 |            | or    | 000000 | 100101 | 1   | 0     | 0         | 1      | x       | x       | 1         | 1        | 1   | 0   | 0     | 0      | 10     | 00         | 100   |
| 35 | 111(写回)    | sli   | 001010 |        | 1   | 0     | 0         | 1      | x       | x       | 1         | 1        | 1   | 0   | 0     | 0      | 01     | 00         | 110   |

输出：

● State：当前状态

内部连线：

● Nowstate：当前状态

● Nextstate：下一个状态



模块代码:

```
module control_unit(
    input [5:0] opCode, funct,
    input zero,CLK,reset,
    output reg PCWre, ALUSrcA, ALUSrcB, DBDataSrc, RegWre, WrRegDSrc,
    InsMemRW, mRD, mWR, IRWre, ExtSel,
    output reg [1:0] PCSrc, RegDst,
    output reg [2:0] ALUOp,state
);
    reg [2:0] nowstate;
    reg [2:0] nextstate;

    always@(posedge CLK) begin
        if (reset == 0) begin
            nowstate = 3'b000;
        end
        else begin
            nowstate = nextstate;
        end
        state = nowstate;
    end

    always@(*) begin
        case(nowstate)
            3'b000: nextstate = 3'b001;
            3'b001: begin
                nextstate[0] = (opCode == 6'b000100 || opCode == 6'b000101
                || opCode == 6'b000001) ? 1 : 0;
                nextstate[1] = (opCode == 6'b000100 || opCode == 6'b000101
                || opCode == 6'b000001 ||
                opCode == 6'b000010 || opCode == 6'b000011 ||
                opCode == 6'b111111 ||
                (opCode == 6'b000000 && funct == 6'b001000)) ?
                0 : 1;
                nextstate[2] = (opCode == 6'b000010 || opCode == 6'b000011
                || opCode == 6'b111111 ||
                (opCode == 6'b000000 && funct == 6'b001000)
                || opCode == 6'b101011 || opCode == 6'b100011) ? 0 : 1;
            end
            3'b010: nextstate = 3'b011;
            3'b011: nextstate = (opCode == 6'b100011) ? 3'b100 : 3'b000;
        end
    end
end
```

```

        3'b100: nextstate = 3'b000;
        3'b101: nextstate = 3'b000;
        3'b110: nextstate = 3'b111;
        3'b111: nextstate = 3'b000;
    endcase
end

always@(*) begin
    PCWre = (nowstate == 3'b111 || nowstate == 3'b100 ||
        nowstate == 3'b101 || (nowstate == 3'b011 && opCode ==
6'b101011) ||
        (nowstate == 3'b001 && (opCode == 6'b000010 || opCode ==
6'b000011 ||
        (opCode == 6'b000000 && funct ==
6'b001000)))) ? 1 : 0;
    InsMemRW = 1;
    ALUSrcA = (nowstate == 3'b110 && (opCode == 6'b000000 && funct ==
6'b000000)) ? 1 : 0;
    DBDataSrc = (nowstate == 3'b100 && opCode == 6'b100011) ? 1 : 0;
    RegWre = (nowstate == 3'b111 || nowstate == 3'b100 ||
        (nowstate == 3'b001 && opCode == 6'b000011)) ? 1 : 0;
    WrRegDSrc = (nowstate == 3'b001 && opCode == 6'b000011) ? 0 : 1;
    InsMemRW = 1;
    mRD = 1;
    mWR = (nowstate == 3'b011 && opCode == 6'b101011) ? 1 : 0;
    IRWre = (nowstate == 3'b000) ? 1 : 0;
    ExtSel = (nowstate == 3'b000 || nowstate == 3'b010 || nowstate ==
3'b101 ||
        opCode == 6'b001010 || opCode == 6'b001001) ? 1 : 0;

    ALUSrcB = (nowstate == 3'b010 || (nowstate == 3'b110 &&
        (opCode == 6'b001001 || opCode ==
6'b001010 ||
        opCode == 6'b001100 || opCode ==
6'b001101 ||
        opCode == 6'b001110)))) ? 1 : 0;

    case(opCode)
        6'b000011: RegDst = 2'b00;
        6'b000000: RegDst = 2'b10;
        default: RegDst = 2'b01;
    endcase
    case(nowstate)
        3'b010: ALUOp = 3'b000;
        3'b101: ALUOp = 3'b001;

```

```

        3'b110: begin
            ALUOp[0] = ((opCode == 6'b000000 && (funct == 6'b100010 ||
funct == 6'b101010 || funct == 6'b100101)) ||
                        opCode == 6'b001101 || opCode == 6'b001110) ? 1 :
0;
            ALUOp[1] = (opCode == 6'b001101 || opCode == 6'b001110 ||
opCode == 6'b001010 ||
                        (opCode == 6'b000000 && (funct == 6'b100101 ||
funct == 6'b000000))) ? 1 : 0;
            ALUOp[2] = (opCode == 6'b001100 || opCode == 6'b000001 ||
                        (opCode == 6'b000000 && (funct == 6'b100100 ||
funct == 6'b101010)) ||
                        opCode == 6'b001010) ? 1 : 0;
        end
    endcase
end

always@(*) begin
    case(opCode)
        6'b000010: PCSrc = 2'b11;
        6'b000011: PCSrc = 2'b11;
        6'b000000: PCSrc = (funct == 6'b001000) ? 2'b10 : 2'b00;
        6'b000100: PCSrc = (nowstate == 3'b101 && zero == 0) ? 2'b01 :
2'b00;
        6'b000101: PCSrc = (nowstate == 3'b101 && zero == 1) ? 2'b01 :
2'b00;
        6'b000001: PCSrc = (nowstate == 3'b101 && zero == 1) ? 2'b01 :
2'b00;
        default: PCSrc = 2'b00;
    endcase
end
endmodule

```

仿真代码:

```

module control_unit_sim;

    // Declare inputs and outputs
    reg [5:0] opCode, funct;
    reg zero, CLK, reset;
    wire PCWre, ALUSrcA, ALUSrcB, DBDataSrc, RegWre, WrRegDSrc, InsMemRW,
mRD, mWR, IRWre, ExtSel;
    wire [1:0] PCSrc, RegDst;
    wire [2:0] ALUOp, state;

```

```

// Instantiate the control unit module
control_unit uut (
    .opCode(opCode),
    .funct(funct),
    .zero(zero),
    .CLK(CLK),
    .reset(reset),
    .PCWre(PCWre),
    .ALUSrcA(ALUSrcA),
    .ALUSrcB(ALUSrcB),
    .DBDataSrc(DBDataSrc),
    .RegWre(RegWre),
    .WrRegDSrc(WrRegDSrc),
    .InsMemRW(InsMemRW),
    .mRD(mRD),
    .mWR(mWR),
    .IRWre(IRWre),
    .ExtSel(ExtSel),
    .PCSrc(PCSrc),
    .RegDst(RegDst),
    .ALUOp(ALUOp),
    .state(state)
);

// Clock generation
always begin
    #5 CLK = ~CLK; // Generate clock with a period of 10 time units
end

// Initial block for simulation
initial begin
    // Initialize all inputs
    CLK = 0;
    reset = 0;
    opCode = 6'b000000; // example opCode
    funct = 6'b000000; // example funct
    zero = 0;

    // Test reset condition
    reset = 1;
    #10 reset = 0; // Deassert reset

    // Test case 1: Example with a specific opCode and funct

```

```

opCode = 6'b000000; // Example opCode (R-type instruction)
funct = 6'b100000; // Example funct (add instruction)
#10;

// Test case 2: Branch instruction (beq)
opCode = 6'b000100; // opCode for beq
funct = 6'b000000;
#10;

// Test case 3: Jump instruction
opCode = 6'b000010; // opCode for jump
#10;

// Test case 4: Load instruction (lw)
opCode = 6'b100011; // opCode for lw
#10;

// Test case 5: Store instruction (sw)
opCode = 6'b101011; // opCode for sw
#10;

// Test case 6: Branch not equal instruction (bne)
opCode = 6'b000101; // opCode for bne
zero = 1; // zero flag set for bne
#10;

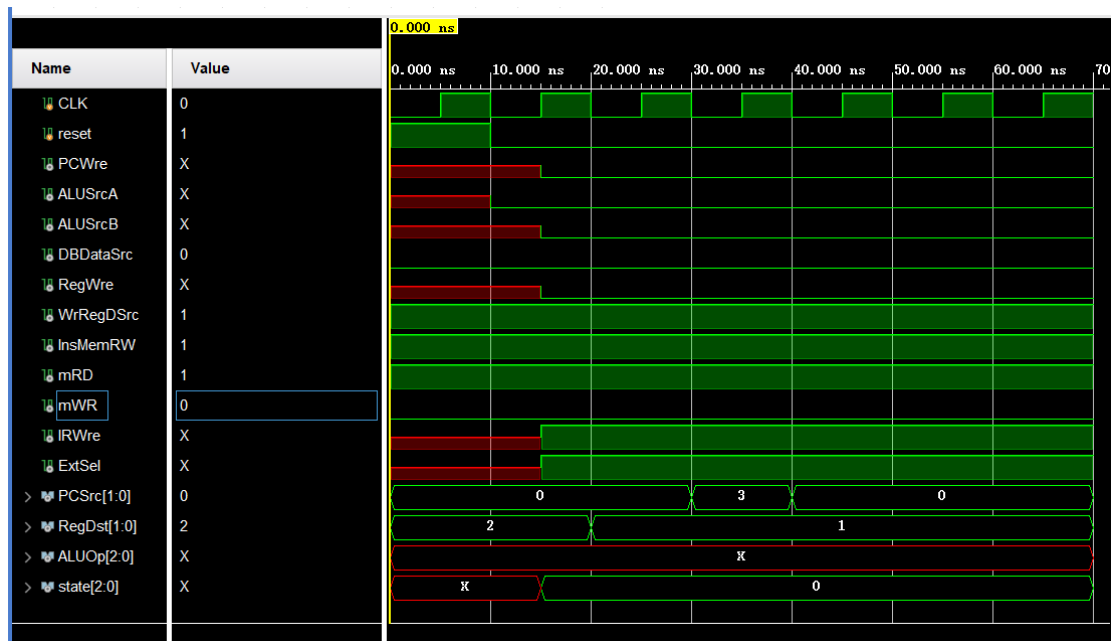
// End simulation
$finish;
end

// Monitor the outputs to track the signals in simulation
initial begin
    $monitor("Time: %0t, opCode: %b, funct: %b, zero: %b, PCWre: %b,
ALUSrcA: %b, ALUSrcB: %b, DBDataSrc: %b, RegWre: %b, WrRegDSrc: %b,
InsMemRW: %b, mRD: %b, mWR: %b, IRWre: %b, ExtSel: %b, PCSrc: %b, RegDst: %b,
ALUOp: %b, state: %b",
            $time, opCode, funct, zero, PCWre, ALUSrcA, ALUSrcB,
            DBDataSrc, RegWre, WrRegDSrc, InsMemRW, mRD, mWR, IRWre, ExtSel, PCSrc,
            RegDst, ALUOp, state);
end

endmodule

```

仿真波形:



## 2. ALU

ALU 是 CPU 中的负责逻辑运算的模块。该模块接受 5 个输入：

- ra : 从 rs 寄存器读取的数据
- rb: 从 rt 寄存器读取的数据
- immediateExt: 从符号拓展单元读取的数据
- shamt: 直接从指令中解码得到的 Shame 字段
- ALUop : 从控制单元读取的 ALU 控制信号，根据这个决定 ALU 的运算功能
- ALUSrcA : 决定进行计算的数据是 ReadData1 还是 Sa
- ALUSrcB : 决定进行计算的数据是 ReadData2 还是 Ext

该模块输出三个信号：

- zero : 输出为 0 时为 1，否则为 0
- sign: 输出为 1 为负数
- result : 进行运算后的结果

模块代码:

```
module ALU(..
    input [31:0]ra,
    input [31:0]rb,
    input [4:0]shamt,
    input [31:0]immediateExt,
    input [2:0]ALUOp,
    input ALUSrcA,//选择器, 控制信号
    input ALUSrcB,

    output reg[31:0]result,
    // output sign,
    output zero
);

    wire [31:0]inputa;
    wire [31:0]inputb;

    assign inputa=ALUSrcA? {{27{1'b0}},shamt}:ra;
    assign inputb=ALUSrcB? immediateExt : rb;
    // assign sign=result<0 ? 1:0;//非负数为 0, 负数 1
    assign zero=(result==32'b0)? 1:0;

    always@(*)begin
        case(ALUOp)
            3'b000://+
                result=inputa+inputb;
            3'b001://-
                result=inputa-inputb;
            3'b010://<<
                result=inputb<<inputa;
            3'b011:
                result=inputa|inputb;
            3'b100:
                result=inputa&inputb;
            3'b101:
                result=inputa<inputb?1:0;
            3'b110:
                result=(((inputa< inputb) && ( inputa[31] ==
inputb[31] )) || ( ( inputa[31] ==1 && inputb[31] == 0))) ? 1:0;
            3'b111:
```

```

        result=inputa^~inputb;
    default:
        result = 32'h0000;
    endcase
end
endmodule

```

仿真代码:

```

module tb_ALU;

    // 输入信号
    reg [31:0] ra;
    reg [31:0] rb;
    reg [4:0] shamt;
    reg [31:0] immediateExt;
    reg [2:0] ALUOp;
    reg ALUSrcA;
    reg ALUSrcB;

    // 输出信号
    wire [31:0] result;
    wire sign;
    wire zero;

    // 实例化 ALU 模块
    ALU uut (
        .ra(ra),
        .rb(rb),
        .shamt(shamt),
        .immediateExt(immediateExt),
        .ALUOp(ALUOp),
        .ALUSrcA(ALUSrcA),
        .ALUSrcB(ALUSrcB),
        .result(result),
        .sign(sign),
        .zero(zero)
    );

    // 仿真过程
    initial begin
        // 测试初始化
        ra = 32'b00000000000000000000000000000001; // 1
        rb = 32'b00000000000000000000000000000010; // 2
        shamt = 5'b00001; // 左移的位数
    end
endmodule

```



```

    immediateExt = 32'b00000000000000000000000000000100; // 4

    ALUSrcA = 0; ALUSrcB = 0; ALUOp = 3'b000; // 加法
    #10;

    ALUOp = 3'b001; // 减法
    #10;

    ALUOp = 3'b010; // 左移
    #10;

    ALUOp = 3'b011; // OR
    #10;

    ALUOp = 3'b100; // AND
    #10;

    ALUOp = 3'b101; // 小于等于比较
    #10;

    ALUOp = 3'b110; // 小于比较（有符号数比较）
    #10;

    ALUOp = 3'b111; // XNOR
    #10;

    // 测试 ALUSrcA 和 ALUSrcB 控制信号
    ALUSrcA = 1; ALUSrcB = 1; ALUOp = 3'b000; // 加法，使用 shamt 和
immediateExt
    #10;

    ALUSrcA = 0; ALUSrcB = 1; ALUOp = 3'b010; // 左移，使用
immediateExt
    #10;

    // 停止仿真
    $finish;

end

// 观察信号
initial begin
    $monitor("Time = %0t, ra = %d, rb = %d, shamt = %b, immediateExt
= %d, ALUOp = %b, result = %d, sign = %b, zero = %b",
            $time, ra, rb, shamt, immediateExt, ALUOp, result, sign,

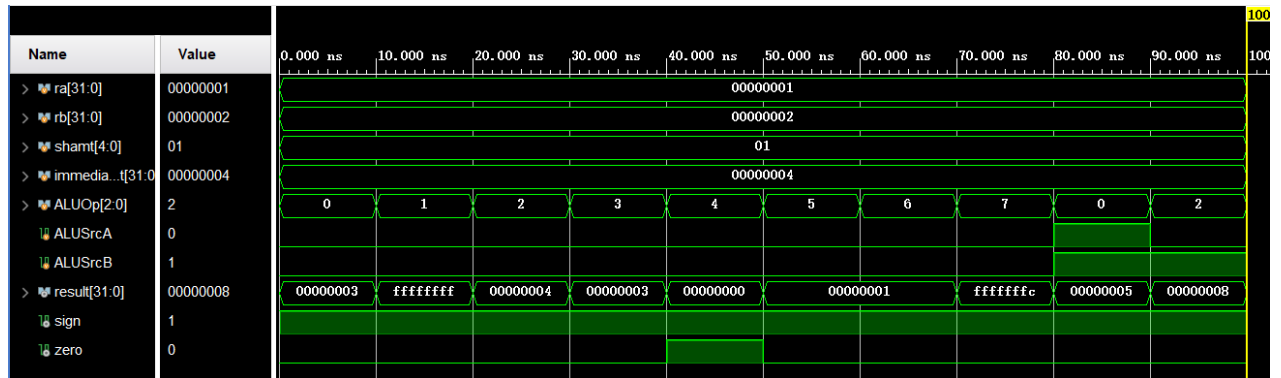
```

```

zero);
    end
endmodule

```

输出波形：



### 3. DataMemory

DataMemory 用于将数据存储到内存中和从内存中读取数据，该模块

接受 4 个输入：

- CLK : 时钟信号，只允许在时钟下降沿进行写操作
- raAddIm: 读取或写数据的地址
- datain : 要进行写操作的数据

模块代码：

```

module DataMemory (//mRD,mWR
    input CLK,
    input mWR,//写使能 0 写
    input mRD,//读 0
    input wire [31:0]raAddIm,//lw,sw 指令中输入的为 ra+im,还没有乘以四,
    输入到 address,cpu 里应该是 result 吧
    input wire [31:0]datain,//sw 时使用，输入为要写入的寄存器的地址，应
    该是 rs，所以接口应该是 ra，好像那个实验报告有问题

```

```

        output wire[31:0]dataout
    );
    //内存体
    reg [7:0]mem[0:127]; //按字节编址, 128 个字节
    wire[31:0]address;

    assign address=(raAddIm<<2); // *4

    //read, 这是大端吗

    always@(negedge CLK) begin
        if(mWR==1) begin
            mem[address]<=datain[31:24];
            mem[address+1]<=datain[23:16];
            mem[address+2]<=datain[15:8];
            mem[address+3]<=datain[7:0];
        end
    end

    assign dataout[7:0] = mRD==1 ? mem[address+3]:8'bz;
    assign dataout[15:8]= mRD==1 ? mem[address+2]:8'bz;
    assign dataout[23:16]= mRD==1 ? mem[address+1]:8'bz;
    assign dataout[31:24]= mRD==1 ? mem[address]:8'bz;
endmodule

```

## 仿真代码:

```

module tb_DataMemory;
    reg CLK;
    reg mWR;
    reg mRD;
    reg [31:0] raAddIm; // 地址输入
    reg [31:0] datain; // 写入数据
    wire [31:0] dataout; // 读取数据

    // 实例化 DataMemory 模块
    DataMemory uut (
        .CLK(CLK),
        .mWR(mWR),
        .mRD(mRD),
        .raAddIm(raAddIm),
        .datain(datain),

```

```

        .dataout(dataout)
    );

    // 时钟生成
    always begin
        CLK = ~CLK;
        #5; // 时钟周期 10ns
    end

    initial begin
        // 初始化信号
        CLK = 0;
        mWR = 1; // 1 表示不写入
        mRD = 1; // 1 表示不读取
        raAddIm = 32'b0;
        datain = 32'b0;

        // 等待系统稳定
        #10;

        // 测试写操作
        // 写入数据 0x12345678 到地址 4
        raAddIm = 32'd1; // 地址=ra+im, 这里设置为 1 表示地址 4
        datain = 32'h12345678;
        mWR = 0; // 使能写入
        mRD = 1; // 不读取
        #10; // 等待一个时钟周期

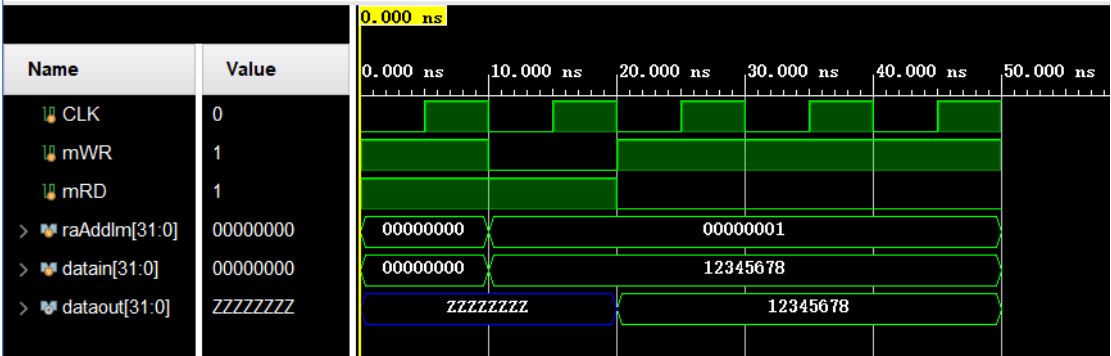
        // 测试读操作
        // 读取地址 4 的数据
        raAddIm = 32'd1; // 地址=ra+im, 这里设置为 1 表示地址 4
        mWR = 1; // 不写入
        mRD = 0; // 使能读取
        #10; // 等待一个时钟周期

        // 打印输出
        $display("Read data: %h", dataout);

        // 结束仿真
        #20;
        $finish;
    end
end
endmodule

```

仿真波形：



#### 4. Instuction\_Reg 和 IR\_Reg

指令存储器模块接受来自 PC 的地址并从存储器中取出指令并进行解码，然后输出解码后的字段。IR\_Reg 将

模块代码：

```
module Instuction_Reg(  
    input IR_CLK,RW,  
    input [31:0] address,datain,  
    output reg [31:0] dataout  
);  
  
    reg [31:0] data;  
    reg [7:0] mem [0:127];  
  
    initial begin  
        $readmemb("C:/Users/zhang/Desktop/Instruction2.txt", mem); // 从文件中读取指令集  
        dataout = 0;  
    end  
  
    always @(address or RW)  
    if (RW == 1) begin  
        data[31:24] = mem[address];  
        data[23:16] = mem[address + 1];  
        data[15:8] = mem[address + 2];  
        data[7:0] = mem[address + 3];  
    end  
end
```

```

        always @(negedge IR_CLK) begin
            dataout <= data;
        end

endmodule

module IR_Reg(
    input CLK,RW,
    input [31:0] datain,
    output reg [31:0] dataout,
    output reg [5:0] opCode,funct,
    output reg [4:0] rs,rt,rd,shamt,
    output reg [15:0] immediate,
    output reg [25:0] target
);

    always @(posedge CLK) begin
        if (RW == 1) begin
            dataout = datain;
            opCode = datain[31:26];
            rs = datain[25:21];
            rt = datain[20:16];
            rd = datain[15:11];
            shamt = datain[10:6];
            funct = datain[5:0];
            immediate = datain[15:0];
            target = datain[25:0];
        end
    end

end

endmodule

```

仿真代码:

```

module tb_Instruction_Reg;
    // 定义信号
    reg IR_CLK;
    reg RW;
    reg [31:0] address;
    reg [31:0] datain;
    wire [31:0] dataout;

    // 实例化被测试模块

```

```

Instuction_Reg uut (
    .IR_CLK(IR_CLK),
    .RW(RW),
    .address(address),
    .datain(datain),
    .dataout(dataout)
);

// 时钟生成过程
always begin
    #5 IR_CLK = ~IR_CLK; // 产生周期为 10 的时钟信号
end

initial begin
    // 初始化信号
    IR_CLK = 0;
    RW = 0;
    address = 0;
    datain = 0;

    // 打开文件，供仿真观察输出
    $monitor("At time %t, address = %h, RW = %b, dataout = %h", $time,
address, RW, dataout);

    // 读取内存中的数据
    // 模拟从内存中读取数据
    #10 address = 32'h0; RW = 1; // 地址 0 开始读取
    #10 address = 32'h4; RW = 1; // 地址 4 开始读取
    #10 address = 32'h8; RW = 1; // 地址 8 开始读取

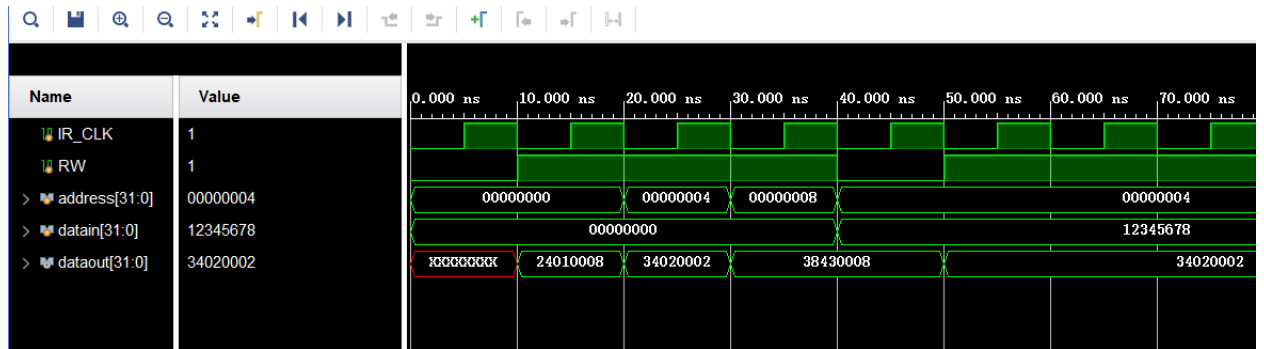
    // 测试读写操作
    #10 address = 32'h4; RW = 0; datain = 32'h12345678; // 写入数据
    #10 address = 32'h4; RW = 1; // 读取刚才写入的数据

    // 结束仿真
    #50 $finish;
end

endmodule

```

仿真波形：



## 5. PC 和 PCAdd

PC 模块接受一个控制信号，决定将 PC+4 还是 PC+4+Immedate 还是 jumppc, jrpc (out1) 送入到指令存储器进行指令读取。

```
module pc( // PCWre, PCSrc, Reset 我没写
    input [1:0] PCSrc,
    input CLK,
    input PCWre,
    input [31:0] immediate,
    input [31:0] jumppc,
    input [31:0] out1,
    input Reset,

    output reg [31:0] currentPC// 当前 pc
    // output [31:0] nextpc,
    // output [3:0] pc4
);
    // 计算下一个 PC 值
    //assign nextpc = PCSrc[0] ? (currentPC + (immediate << 2)) :
        // (PCSrc[1] ? jumppc : currentPC + 4);
    //assign pc4 = currentPC[31:28];
    initial begin
        currentPC = 0;
    end

    always @(negedge CLK or negedge Reset) begin
        if (Reset == 0)
            currentPC <= 0;
        else if (PCWre == 1) begin // pc 可写
```



```

        if (PCSrc == 2'b00)          // pc+4
            currentPC <= currentPC + 4;
        else if (PCSrc == 2'b01)     // branch
            currentPC = currentPC + (immediate << 2) + 4;
        else if (PCSrc == 2'b10)
            currentPC = out1;
        else if (PCSrc == 2'b11)     // jump
            currentPC = jumppc;
    end
end
endmodule

```

```

module PCAdd(
    input [25:0] target,
    input [31:0] currentPC,
    output reg[31:0] jumppc
);
    always@(*)begin
        jumppc<={currentPC[31:28],target[25:0],2'b00};
    end
endmodule

```

仿真代码:

```

module tb_pc;

    // 定义信号
    reg [1:0] PCSrc;
    reg CLK;
    reg PCWre;
    reg [31:0] immediate;
    reg [31:0] jumppc;
    reg [31:0] out1;
    reg Reset;
    wire [31:0] currentPC;

    // 实例化被测试模块
    pc uut (
        .PCSrc(PCSrc),
        .CLK(CLK),
        .PCWre(PCWre),
        .immediate(immediate),

```

```

        . jumppc(jumppc),
        . out1(out1),
        . Reset(Reset),
        . currentPC(currentPC)
    );

    // 时钟生成过程
    always begin
        #5 CLK = ~CLK; // 产生周期为 10 的时钟信号
    end

    initial begin
        // 初始化信号
        CLK = 0;
        PCWre = 0;
        PCSrc = 2'b00;
        immediate = 32'h00000004;
        jumppc = 32'h10000000;
        out1 = 32'h20000000;
        Reset = 0;

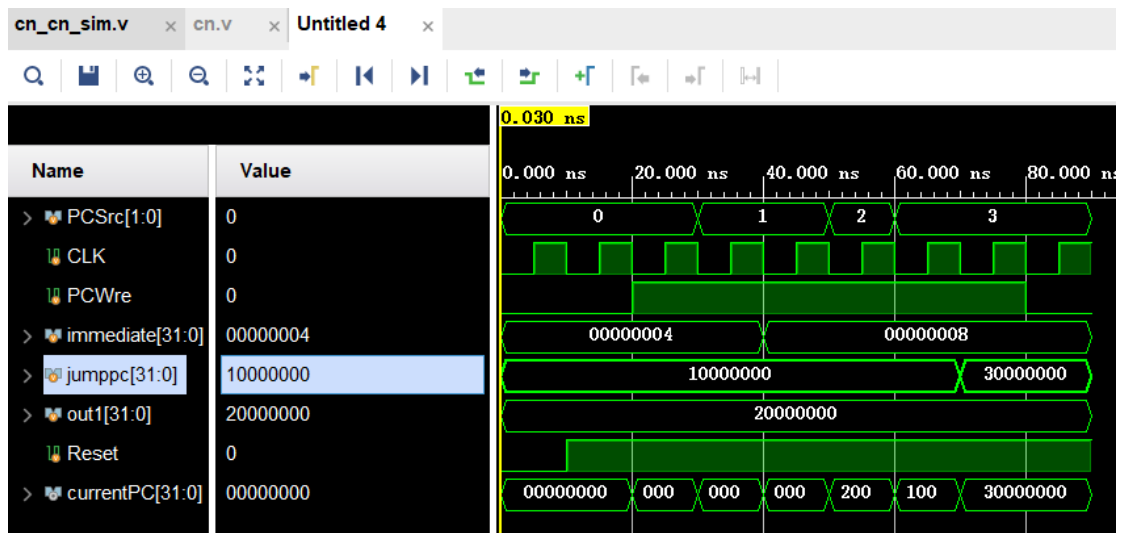
        // 打开文件，供仿真观察输出
        $monitor("At time %t, currentPC = %h, PCSrc = %b, PCWre = %b, Reset
= %b, immediate = %h, jumppc = %h, out1 = %h",
            $time, currentPC, PCSrc, PCWre, Reset, immediate, jumppc,
            out1);

        // 仿真过程
        #10 Reset = 1; // 激活 Reset
        #10 PCWre = 1; PCSrc = 2'b00; // PC+4
        #10 PCSrc = 2'b01; // Branch (地址 + immediate)
        #10 immediate = 32'h00000008; // 设置一个新的 immediate, 地址偏移量
        #10 PCSrc = 2'b10; // 使用 out1
        #10 PCSrc = 2'b11; // 跳转到 jumppc
        #10 jumppc = 32'h30000000; // 设置新的跳转地址
        #10 PCWre = 0; // 不再写入
        #10 $finish; // 结束仿真
    end

endmodule

```

仿真波形:



## 6.RegFile 和 reg\_outA,reg\_outB, selectAM

寄存器组负责从寄存器的读写操作，该模块接受 9 个输入：

- CLK：时钟信号，当时钟下降沿时才能进行写操作
- RegDst：RegDst 为真时，处理 R 型指令，rd 为目标操作数寄存器，为假时处理 I 型指令，rt 为目标操作数寄存器
- RegWre：为 1 时才允许进行写操作
- DBDataSrc：选择写的数据是来自 ALU 还是数据存储器
- rs：rs 寄存器地址
- rt：rt 寄存器地址
- rd：rd 寄存器地址
- datain：来自 selectAM 的输出

该模块输出为：

- busA：输出的第一个数
- busB：输出的第二个数
- W\_data：要进行写的数，放在输出端口为了在 7 段数码管上

显示

而 reg\_outA,reg\_outB 目的是把 busA,busB 暂存到下一个时钟周期再读出。

SelectAM:

- dataFromALU: 来自 ALU 输出的数据
- dataFromM: 来自数据存储器的数据
- dataout: 选择输出

模块代码:

```
module RegFile(//控制信号: RegWre, RegDst, DBDataSrc
    input CLK ,RegWre, DBDataSrc,
    input [1:0]RegDst,
    input [4:0] rs, rt, rd,
    //input [31:0] dataFromALU, dataFromM,
    input [31:0]datain,
    input [31:0]pc,
    output reg[31:0] busA,busB,
    output [31:0] W_data
);
```

```
    reg [4:0]writeReg;//写入寄存器的编号
    wire [31:0]m_data;
    reg [31:0] register[0:31];
    integer i;
    initial begin
        for(i = 0;i < 32; i=i+1 )
            register[i] = 32'b0;
    end
    //判断指令类型
    assign m_data= DBDataSrc==1?datain:pc;
    assign W_data  =m_data;
    always @(negedge CLK) begin
        case(RegDst)
            2'b00:writeReg=5'b11111;
            2'b01:writeReg=rt;
            2'b10:writeReg=rd;
        endcase
    end
```

```

        if (RegWre == 1)
            register[writeReg] <= m_data;
            busA <= register[rs];
            busB <= register[rt];
        end
    endmodule

module Reg(
    input wire[31:0] in,
    input wire clk,
    output reg[31:0] out
);
    always @(posedge clk) begin
        out <= in;
    end
endmodule

module selectAM(
    input [31:0] dataFromALU, dataFromM,
    input DBDataSrc,
    output wire[31:0] dataout
);

    assign dataout=(DBDataSrc)?dataFromM:dataFromALU;
endmodule、

```

仿真代码：

```

module tb_RegFile;
    // 定义信号
    reg CLK;
    reg RegWre;
    reg DBDataSrc;
    reg [1:0] RegDst;
    reg [4:0] rs, rt, rd;
    reg [31:0] datain;
    reg [31:0] pc;
    wire [31:0] busA, busB;
    wire [31:0] W_data;

    // 实例化被测试模块
    RegFile uut (
        .CLK(CLK),

```

```

        .RegWre(RegWre),
        .DBDataSrc(DBDataSrc),
        .RegDst(RegDst),
        .rs(rs),
        .rt(rt),
        .rd(rd),
        .datain(datain),
        .pc(pc),
        .busA(busA),
        .busB(busB),
        .W_data(W_data)
    );

// 时钟生成过程
always begin
    #5 CLK = ~CLK; // 产生周期为 10 的时钟信号
end

initial begin
    // 初始化信号
    CLK = 0;
    RegWre = 0;
    DBDataSrc = 0;
    RegDst = 2'b00;
    rs = 5'b00000;
    rt = 5'b00001;
    rd = 5'b00010;
    datain = 32'h12345678;
    pc = 32'hABCDE123;

    // 打开文件，供仿真观察输出
    $monitor("At time %t, busA = %h, busB = %h, W_data = %h,
register[writeReg] = %h",
            $time, busA, busB, W_data, uut.register[rd]);

// 仿真过程
#10 RegWre = 1; RegDst = 2'b00; // 将写入寄存器 31 (RegDst =
2'b00)
#10 RegDst = 2'b01; rt = 5'b00011; // 写入寄存器 3 (RegDst =
2'b01)
#10 RegDst = 2'b10; rd = 5'b01000; // 写入寄存器 8 (RegDst =
2'b10)

#10 DBDataSrc = 1; // 数据源选择为 `datain`
#10 datain = 32'h98765432; // 改变 `datain` 数据

```

```

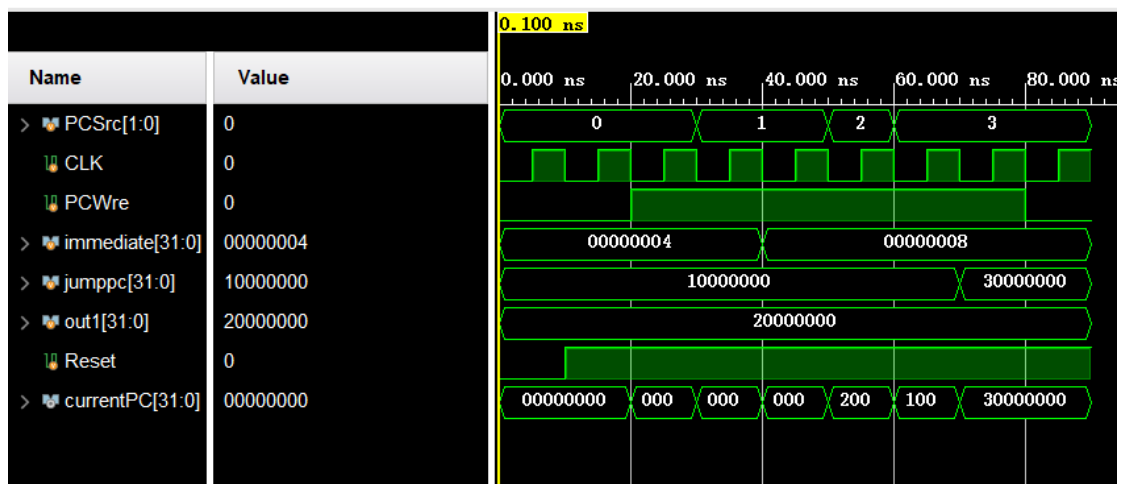
#10 RegWre = 0; // 禁止写寄存器
#10 rs = 5'b00011; // 设置 rs 为 3, 读取寄存器 3
#10 rt = 5'b01000; // 设置 rt 为 8, 读取寄存器 8

// 结束仿真
#50 $finish;
end

endmodule

```

仿真波形：



## 7. SignZeroExt

符号拓展单元对 16 位立即数按符号拓展为 32 位的数据，用于跳转指令和 I 型指令。

输入：

- **immediate** : 所要拓展的 16 位立即数
- **ExtSel** : 为 1 高位补符号位，为 0 高位补 0

输出：

- **ext**: 32 位扩展后的立即数

模块代码:

```
module SignZeroExt(//控制信号: Extsel
    input  [15:0] immediate,
    input  Extsel, //lsign00
    output [31:0] ext
);

    assign ext[15:0] = immediate[15:0];
    assign ext[31:16] = Extsel ? {16{immediate[15]}} : 16'b0;

endmodule
```

仿真代码:

```
module SignZeroExt_tb;

    // 定义信号
    reg [15:0] immediate; // 16 位立即数
    reg Extsel;           // 控制信号
    wire [31:0] ext;      // 扩展后的信号

    // 实例化 SignZeroExt 模块
    SignZeroExt uut (
        .immediate(immediate),
        .Extsel(Extsel),
        .ext(ext)
    );

    // 仿真过程
    initial begin
        // 初始化信号
        immediate = 16'b0000000000000001; // 一个正数
        Extsel = 0; // 零扩展
        #10; // 等待 10 个时间单位
        $display("immediate = %b, Extsel = %b, ext = %b", immediate,
        Extsel, ext);

        immediate = 16'b1111111111111111; // 一个负数
        Extsel = 0; // 零扩展
        #10; // 等待 10 个时间单位
        $display("immediate = %b, Extsel = %b, ext = %b", immediate,
        Extsel, ext);
    end
endmodule
```



```

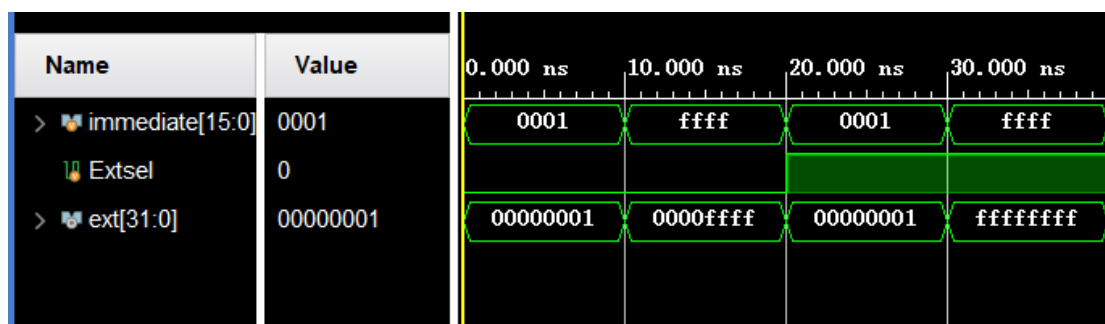
        immediate = 16'b0000000000000001; // 一个正数
        Extsel = 1; // 符号扩展
        #10; // 等待 10 个时间单位
        $display("immediate = %b, Extsel = %b, ext = %b", immediate,
        Extsel, ext);

        immediate = 16'b1111111111111111; // 一个负数
        Extsel = 1; // 符号扩展
        #10; // 等待 10 个时间单位
        $display("immediate = %b, Extsel = %b, ext = %b", immediate,
        Extsel, ext);

        $finish; // 仿真结束
    end
endmodule

```

仿真波形：



## 8. cpu

设计完各个模块之后，接下来的工作是将各个模块的接口连接起来，组合成一个完整的 CPU 模块。

```

module cpu(
    input CLK, reset,
    output [4:0] rs, rt,
    output wire [5:0] opCode,
    output wire [31:0] Out1, Out2, currentPC, Result, DBData,
    output wire[2:0] state
);
    wire PCWre;
    wire [31:0] imext;
    wire [31:0] jumppc;

```

```

wire [31:0] jrpc;
wire [1:0] PCSrc;

wire IRWre;
wire InsMemRW;
wire [31:0] Insdataout;

wire [5:0] funct;
wire [4:0] rd;
wire [15:0] imm;
wire [4:0] shamt;
wire [25:0] target;

wire zero;
wire ALUSrcA;
wire ALUSrcB;
wire DBDataSrc;
wire RegWre;
wire WrRegDSr;
wire mRD;
wire mWR;
wire [1:0] RegDst;
wire [2:0] ALUOp;
wire ExtSel;

wire [31:0] regdatain;

wire [31:0] regoutA;
wire [31:0] regoutB;
wire [31:0] aludata;

wire [31:0] memd;

selectAM se(
    .dataFromALU(Result),
    .dataFromM(memd),
    .DBDataSrc(DBDataSrc),
    .dataout(regdatain)
);

DataMemory mem(
    .mWR(mWR),
    .mRD(mRD),
    .raAddIm(Result),

```

```

        .datain(regoutB),
        .dataout(memd)
    );

    ALU alu(
        .shamt(shamt),
        .immediateExt(imext),
        .ALUSrcA(ALUSrcA),
        .ALUSrcB(ALUSrcB),
        .ra(regoutA), .rb(regoutB), .ALUOp(ALUOp),
        .result(aludata),
        .zero(zero)
    );

    Reg alureg(
        .in(aludata),
        .clk(CLK),
        .out(Result)
    );

    SignZeroExt ex(
        .immediate(imm),
        .Extsel(ExtSel),
        .ext(imext)
    );

    Reg reg_outA(
        .in(Out1),
        .clk(CLK),
        .out(regoutA)
    );

    Reg reg_outB(
        .in(Out2),
        .clk(CLK),
        .out(regoutB)
    );

    RegFile re(
        .DBDataSrc(WrRegDSrc),
        .datain(regdatain),
        .pc(currentPC + 4),
        .CLK(CLK),
        .rs(rs),

```

```

        .rt(rt),
        .rd(rd),
        .RegDst(RegDst),
        .RegWre(RegWre),
        .busA(Out1),
        .busB(Out2),
        .W_data(DBData)
    );

control_unit co(
    .opCode(opCode),
    .funct(funct),
    .zero(zero),
    .CLK(CLK),
    .reset(reset),
    .PCWre(PCWre),
    .ALUSrcA(ALUSrcA),
    .ALUSrcB(ALUSrcB),
    .DBDataSrc(DBDataSrc),
    .RegWre(RegWre),
    .WrRegDSrc(WrRegDSrc),
    .InsMemRW(InsMemRW),
    .mRD(mRD), .mWR(mWR), .IRWre(IRWre), .ExtSel(ExtSel),
    .PCSrc(PCSrc), .RegDst(RegDst), .ALUOp(ALUOp), .state(state)
);

PCAdd pcadd(
    .jumppc(jumppc),
    .currentPC(currentPC),
    .target(target)
);

IR_Reg ir(
    .CLK(CLK),
    .RW(IRWre),
    .datain(Insdataout),
    .opCode(opCode),
    .funct(funct),
    .rs(rs),
    .rt(rt), .rd(rd), .shamt(shamt),
    .immediate(imm),
    .target(target)
);

```

```

    Instuction_Reg in(
        .IR_CLK(CLK),
        .address(currentPC),
        .RW(InsMemRW),
        .dataout(Insdataout)
    );

    pc pc(
        .CLK(CLK),
        .Reset(reset),
        .PCWre(PCWre),
        .PCSrc(PCSrc),
        .immediate(imext),
        .jumppc(jumppc),
        .out1(Out1),
        .currentPC(currentPC)
    );

Endmodule

```

### 仿真代码:

```

module cpu_sim;
    reg CLK;
    reg Reset;
    wire [4:0] rs;
    wire [4:0] rt;
    wire [5:0] opCode;
    wire [2:0] state;
    wire [31:0] Out1, Out2, currentPC, Result, DBData;
    cpu uu(
        .CLK(CLK),
        .reset(Reset),
        .rs(rs), .rt(rt), .opCode(opCode), .state(state), .Out1(Out1), .Out2(Out2), .
currentPC(currentPC), .Result(Result), .DBData(DBData)
    );

    always begin
        CLK = 0;
        #5 CLK = 1'b1;
        Reset=0;
        #5;

        CLK = 0;
    end

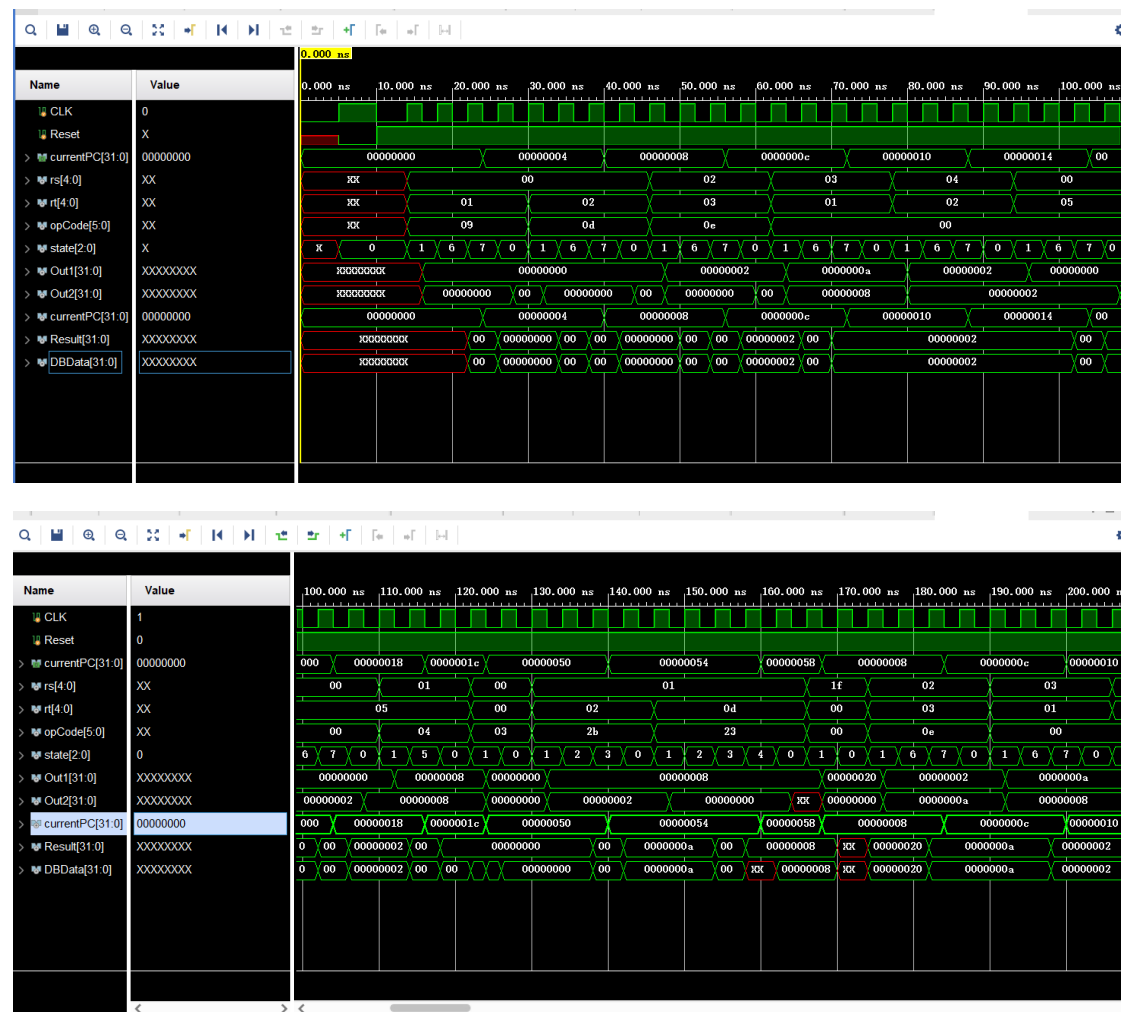
```

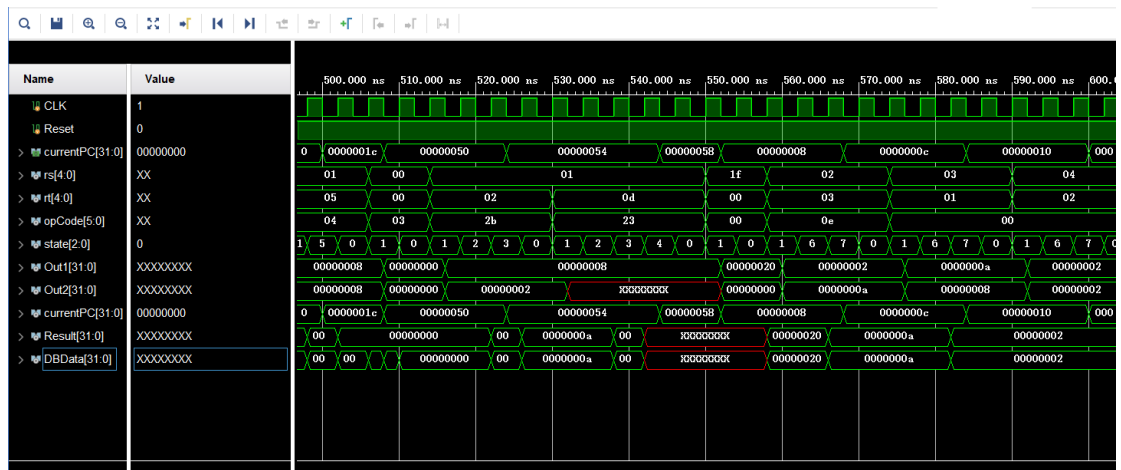
```

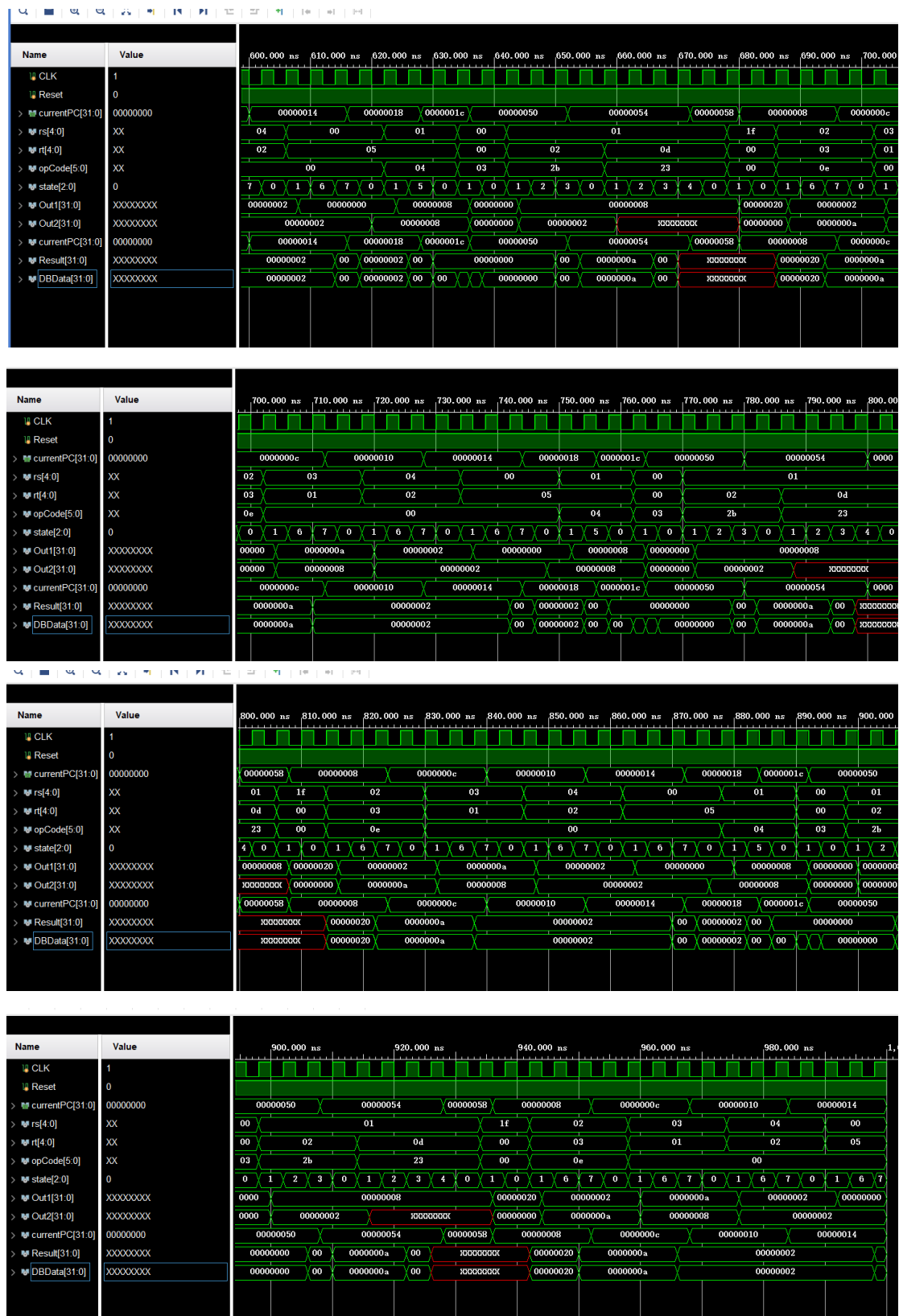
Reset=1;
#2;
forever #2 begin
    CLK = !CLK;
end
end
endmodule

```

仿真波形：







## 10. 进行 Basys 烧写开发板

设计思想：



- 实现 CPU 在板上运行需要两个时钟信号，CPU 工作时钟和 Basys3 板系统时钟。CPU 工作时钟即为按键，是 CPU 正常工作时钟信号，按键必须进行消抖处理；Basys3 板系统时钟即为板提供的正常工作时钟信号，即为 100MHZ。Basys3 板系统时钟信号引脚 对应管脚 W5。
- 每个按键周期，4 个数码管都必须刷新一次。数码管位控信号 AN3-AN0 是 1110、1101、1011、0111，为 0 时点亮该数码管，当然，还应该为数码管各位 “lgfedcba” 引脚输出信号，最高位为 “1”。比如，“当前 PC 值” 低 8 位中的高 4 位和低 4 位，必须经下页转换后送给数码管各引脚。

综上，显示模块大概分为 4 个部分：

- 对 Basys3 板系统时钟信号进行分频，分频的目的用于计数器
- 生成计数器，计数器用于产生 4 个数。这 4 数用于控制 4 个数码管
- 根据计数器产生的数生成数码管相应的位控信号(输出)和接收 CPU 来的相应数据
- 将从 CPU 接收到的相应数据转换为数码管显示信号，再送往数码管显示(输出)

## 分频模块

时钟分频的原理是定义一个计数器 cnt，每当原始时钟（开发板的时钟 BasysCLK）到来时加一。当计数器满  $2^N - 1$  后，再将分频时钟 CLK\_SLow 翻转一次，这样就将 CLK 进行了  $2^N$  分频

```

module CLK_slow(
    input CLK_100mhz, //100mhz 的原始时钟信号

    output reg CLK_slow //输出的分频时钟信号
);

reg [31:0] count = 0; //计数器
reg [31:0] N = 50000; //分频  $2^{19}$ , 得到一个周期大约 5ms 的信号

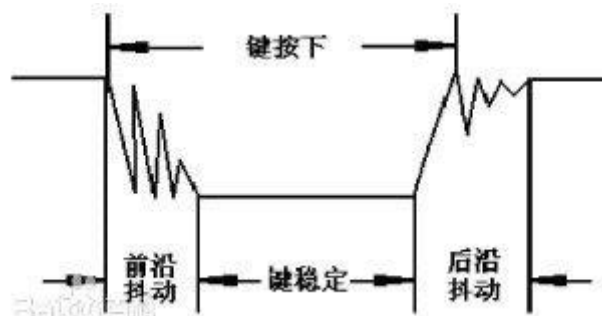
initial CLK_slow = 0;

always @ (posedge CLK_100mhz) begin
    if(count >= N) begin
        count <= 0;
        CLK_slow <= ~CLK_slow;
    end
    else
        count <= count + 1;
end

end
endmodule

```

## 消抖模块



为什么要消抖？见下图：

我们可以看到，在按键按下的一刻，存在一段时间的抖动，同时，在释放按键的一段时间里也是存在抖动的，这就可能导致状态在识别的时候可能检测为多次的按键，因为运行过程中普通的检测一次状态 key 为 1 就执行一次按键操作。所以我们在使用按键时往往需要消抖。

输入消抖模块同样使用了 N 位一个计数器 cnt，当时钟信号到来时，如果按键处于按下状态就将计数器+1，而如果现按键未按下，则清空计数器。将模块输出的防抖按键信号连接到 cnt 第 N 位，这样就保证持续按下  $2^N$  个时钟周期的时间才会输出按键信号，从而屏蔽了短暂的抖动信号。

```
module Keyboard_CLK(  
    input Button,  
    input BasysCLK,  
  
    output CPUCLK  
);  
  
reg button_previous_state;  
reg button_current_state;  
  
wire button_edge;  
  
always @ (posedge BasysCLK) begin  
    button_current_state <= Button;  
    button_previous_state <= button_current_state;  
end  
  
assign button_edge = button_previous_state & (~button_current_state);  
  
reg [20:0] counter;  
  
always @ (posedge BasysCLK) begin  
    if(button_edge)  
        counter <= 21'h0;  
    else  
        counter <= counter + 1;  
end  
  
reg delayed_button_previous_state;  
reg delayed_button_current_state;  
  
//21'h1E8480
```

```

always @ (posedge BasysCLK) begin
    if(counter == 21'h1E8480)
        delayed_button_current_state <= button_current_state;
        delayed_button_previous_state <= delayed_button_current_state;
    end

assign CPUCLK = delayed_button_previous_state &
(~delayed_button_current_state);

endmodule

```

## 显示模块

原理是，当输入一个 4 位的控制信号 display\_data 时，输出一个 8 位的信号，将这个信号输入给 7 段数码管的引脚，就可以让数码管输出对应的数字。

```

module Display_7SegLED(
    input [3:0] display_data,
    output reg [7:0] dispcode
);

always @ (display_data) begin
    case(display_data)
        4'b0000 : dispcode = 8'b1100_0000; //0; '0'-亮灯, '1'-熄灯
        4'b0001 : dispcode = 8'b1111_1001; //1
        4'b0010 : dispcode = 8'b1010_0100; //2
        4'b0011 : dispcode = 8'b1011_0000; //3
        4'b0100 : dispcode = 8'b1001_1001; //4
        4'b0101 : dispcode = 8'b1001_0010; //5
        4'b0110 : dispcode = 8'b1000_0010; //6
        4'b0111 : dispcode = 8'b1101_1000; //7
        4'b1000 : dispcode = 8'b1000_0000; //8
        4'b1001 : dispcode = 8'b1001_0000; //9
        4'b1010 : dispcode = 8'b1000_1000; //A
        4'b1011 : dispcode = 8'b1000_0011; //b
        4'b1100 : dispcode = 8'b1100_0110; //C
        4'b1101 : dispcode = 8'b1010_0001; //d
        4'b1110 : dispcode = 8'b1000_0110; //E
        4'b1111 : dispcode = 8'b1000_1110; //F
        default : dispcode = 8'b0000_0000; //不亮
    endcase
end

```

```

        endcase
    end

endmodule

```

## 选择模块

接受 4 个要显示的数据和一个决定显示哪个数据的控制信号

SelectCode, 然后输出对应的 display\_data 给显示模块

```

module Select(
    input [15:0] In1, In2, In3, In4,
    input [1:0] SelectCode,

    output reg [15:0] DataOut
);

always @ (*) begin
    case (SelectCode)
        2'b00 : DataOut = In1;
        2'b01 : DataOut = In2;
        2'b10 : DataOut = In3;
        2'b11 : DataOut = In4;
    endcase
end

endmodule

```

## 转换模块

对输入的时钟信号和数据进行转换并输出

```

module Transfer(
    input CLK,
    input [15:0] In,

    output reg [3:0] Out,
    output reg [3:0] Bit
);

```

```

integer i;

```

```

initial begin
    i = 0;
end

always @ (negedge CLK) begin
    case(i)
        0 : begin
            Out = In[15:12];
            Bit = 4'b1110;
        end
        1 : begin
            Out = In[11:8];
            Bit = 4'b1101;
        end
        2 : begin
            Out = In[7:4];
            Bit = 4'b1011;
        end
        3 : begin
            Out = In[3:0];
            Bit = 4'b0111;
        end
    endcase
    i = (i == 3) ? 0 : i + 1;
end
endmodule

```

### Basys3 模块：连接所有模块

Basys3 模块连接以上 5 个与显示有关模块和 CPU 模块，最终烧写进开发板里面。

```

module Basys3(
    input CLKButton,
    input BasysCLK,
    input RST_Button,
    input [1:0] SW_in,
    output [7:0] SegOut,
    output [3:0] Bits
);

```

```

    wire cpuCLK;
    wire [31:0] PCdataout;
    wire [31:0] ALUresult;
    wire [5:0] Inop;
    wire [4:0] Inrs, Inrt;
    wire [15:0] Inimmediate;
    wire [31:0] reg_dataA, reg_dataB;
    wire [31:0] DBData;
    wire [2:0] state;

    cpu co(.CLK(cpuCLK),.reset(RST_Button),
    .rs( Inrs),.rt(Inrt),.opCode( Inop),
    .Out1(reg_dataA),.Out2(reg_dataB),.currentPC(PCdataout),.Result(ALUresult),
    .DBData(DBData),.state(state));
    wire Div_CLK;
    CLK_slow clk_slow(
        .CLK_100mhz(BasysCLK),
        .CLK_slow(Div_CLK)
    );

    //Display_7Seg
    wire [3:0] SegIn;

    Display_7SegLED display_led(
        .display_data(SegIn),
        .dispcode(SegOut)
    );

    wire [15:0] display_data;
    Select select(
        .In1({4'b0000,state[2:0],PCdataout[7:0]}),
        .In2({3'b000, Inrs[4:0], reg_dataA[7:0]}),
        .In3({3'b000, Inrt[4:0], reg_dataB[7:0]}),
        .In4({ALUresult[7:0], DBData[7:0]}),
        .SelectCode(SW_in),
        .DataOut(display_data)
    );

    //Display_transfer
    Transfer transfer(
        .CLK(Div_CLK),
        .In(display_data),

```

```

        .Out(SegIn),
        .Bit(Bits)
    );

//keyboard
Keyboard_CLK keyboard(
    .Button(CLKButton),
    .BasysCLK(BasysCLK),
    .CPUCLK(cpuCLK)
);
endmodule

```

同时进行了测试，测试中认为产生按键时钟，测试代码为：

```

module basys_sim(

);
reg CLK, RST;
reg CLKButton;
reg [1:0] SW;

Basys3 test(
    .BasysCLK(CLK),
    .CLKButton(CLKButton),
    .SW_in(SW),
    .RST_Button(RST),

    .SegOut(),
    .Bits()
);
integer i;

initial begin
    //recode
    $dumpfile("Basys3.vcd");
    $dumpvars(0, basys_sim);

    CLK = 0;
    RST = 0;
    CLKButton = 0;
    SW = 2'b00;

    #50;
    RST = 1;

```



```

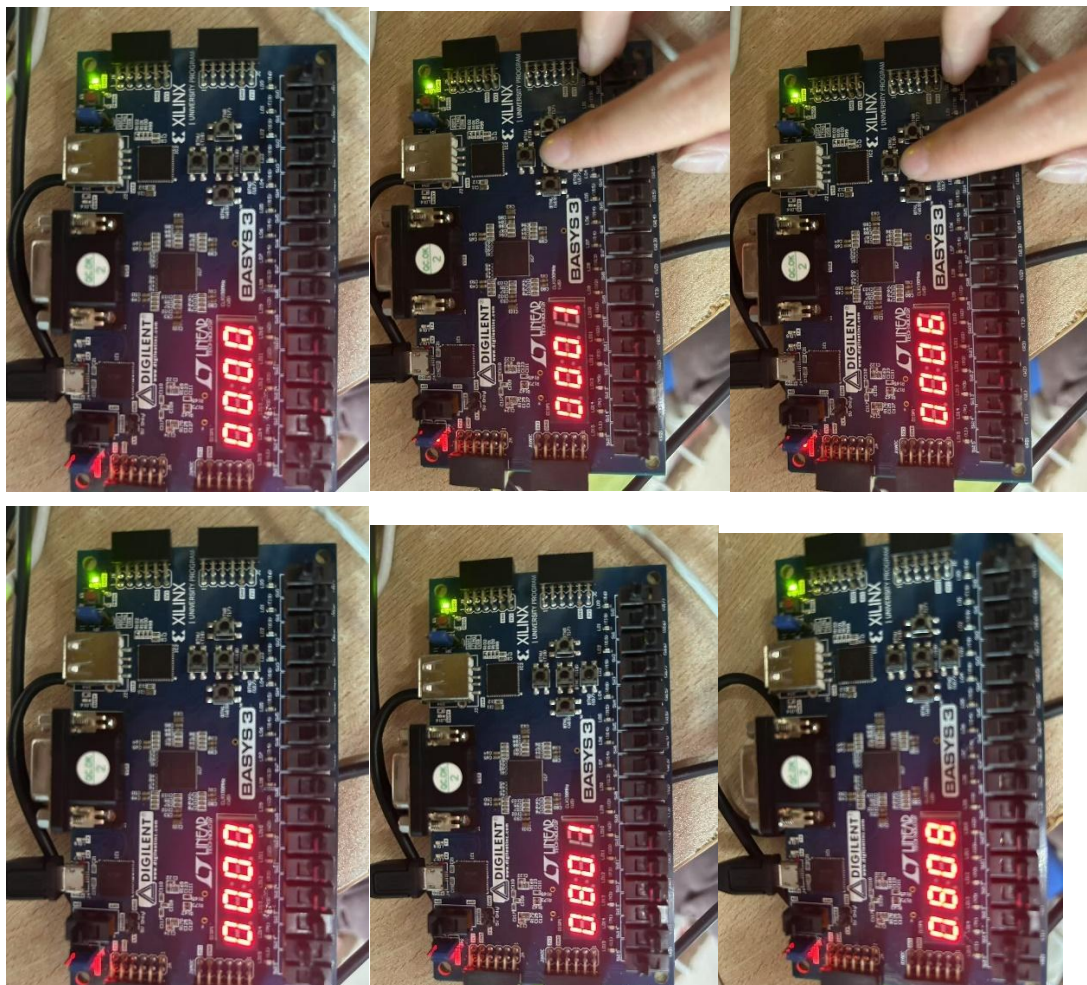
        for (i = 0; i < 1000000; i = i + 1) begin
            #100;
            CLK = ~CLK;
//            #10;
            if (i % 15 == 0) CLKButton = ~CLKButton;
        end
    end
end
endmodule

```

## 11. 烧板

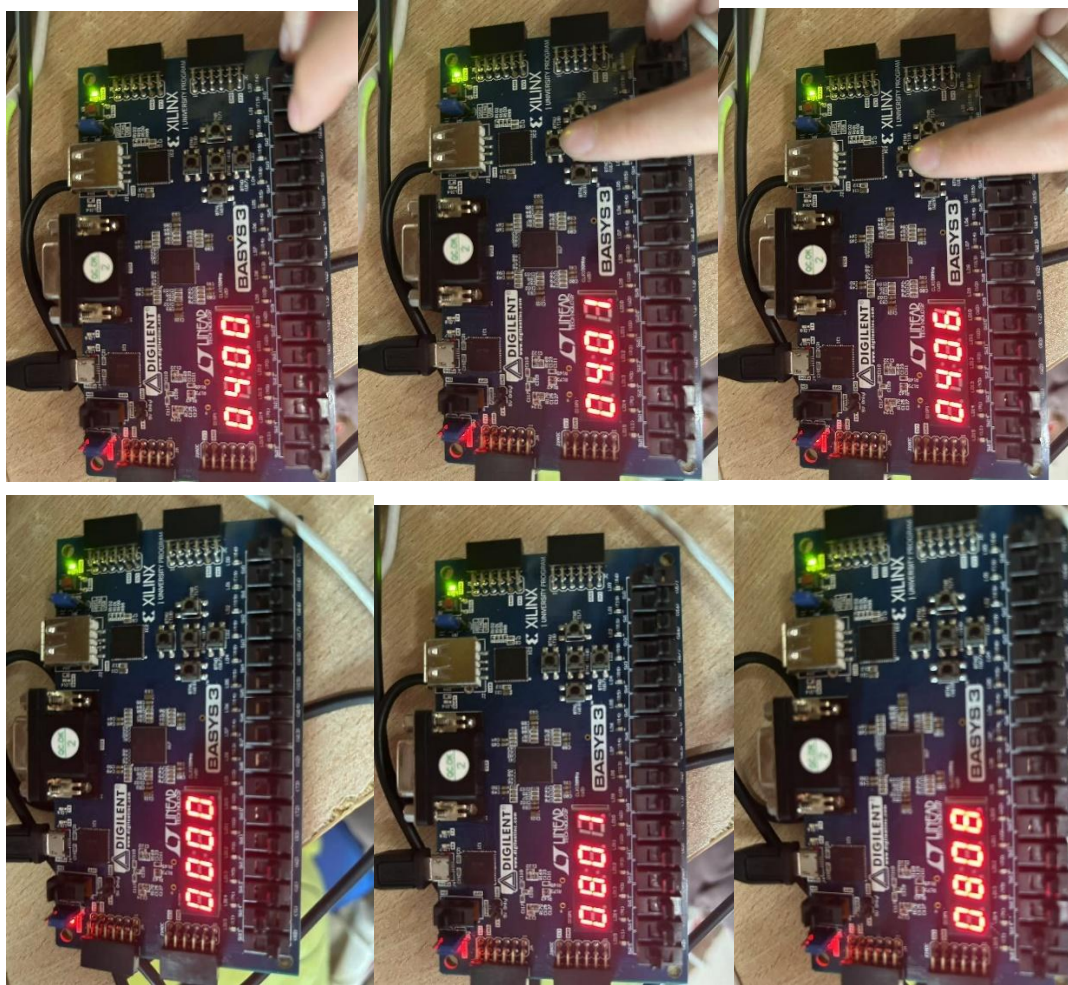
图片：顺序分别为当前指令的 pc 和几个状态，rs 编号和内容，rt 编号和内容，ALU 结果和总线数据。

指令 1:

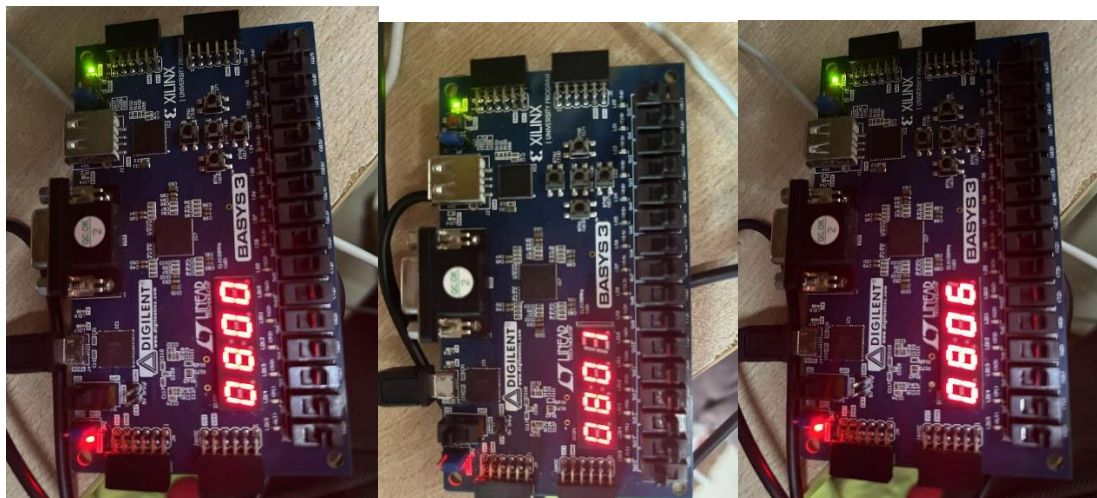




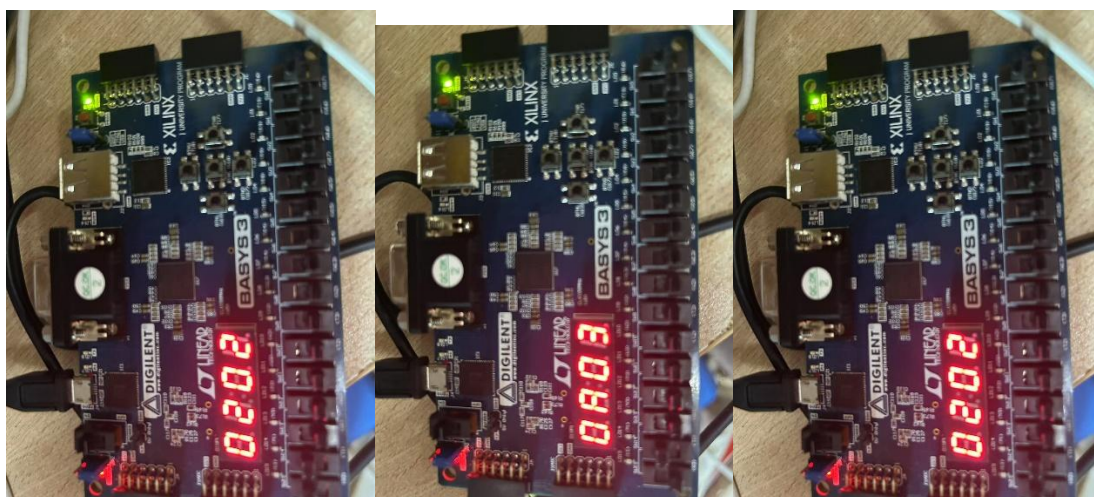
指令 2:



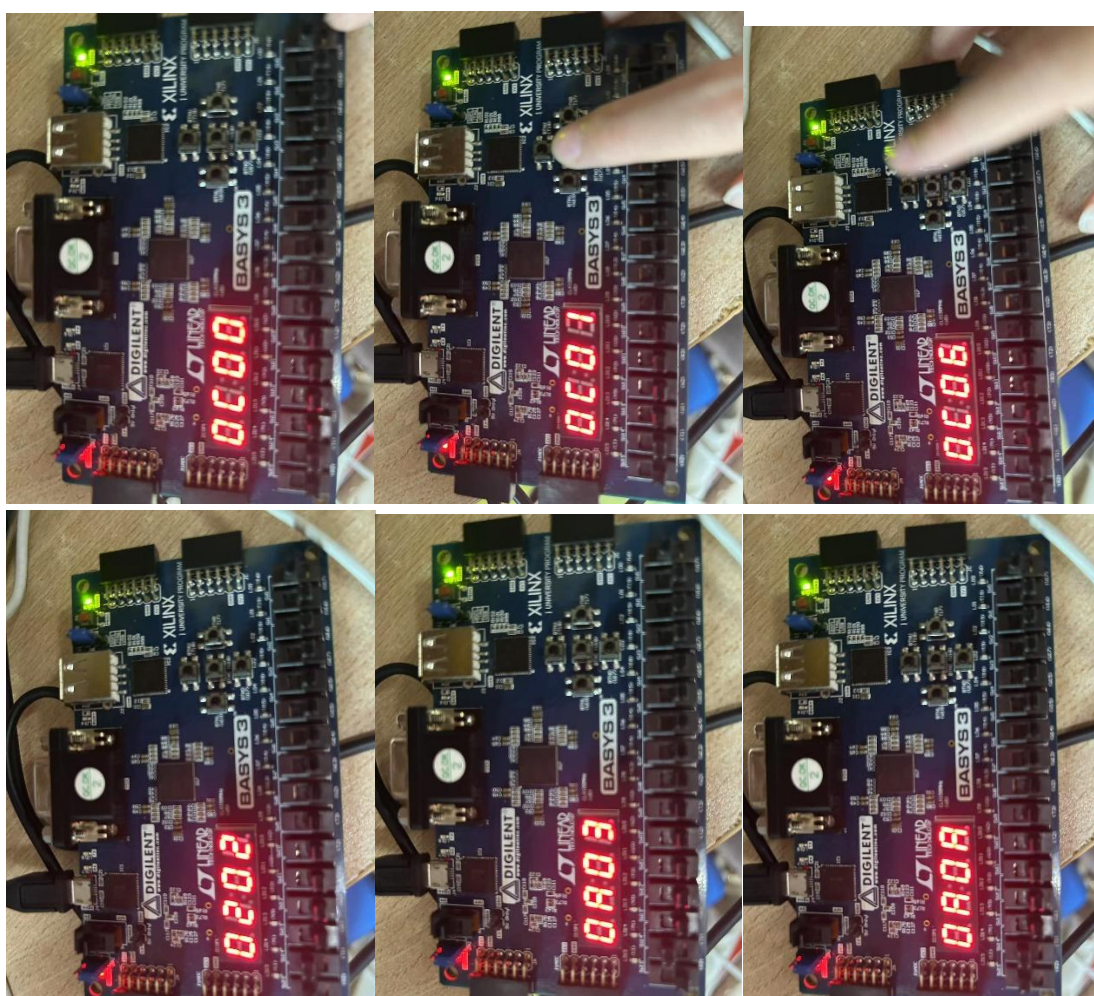
指令 3:



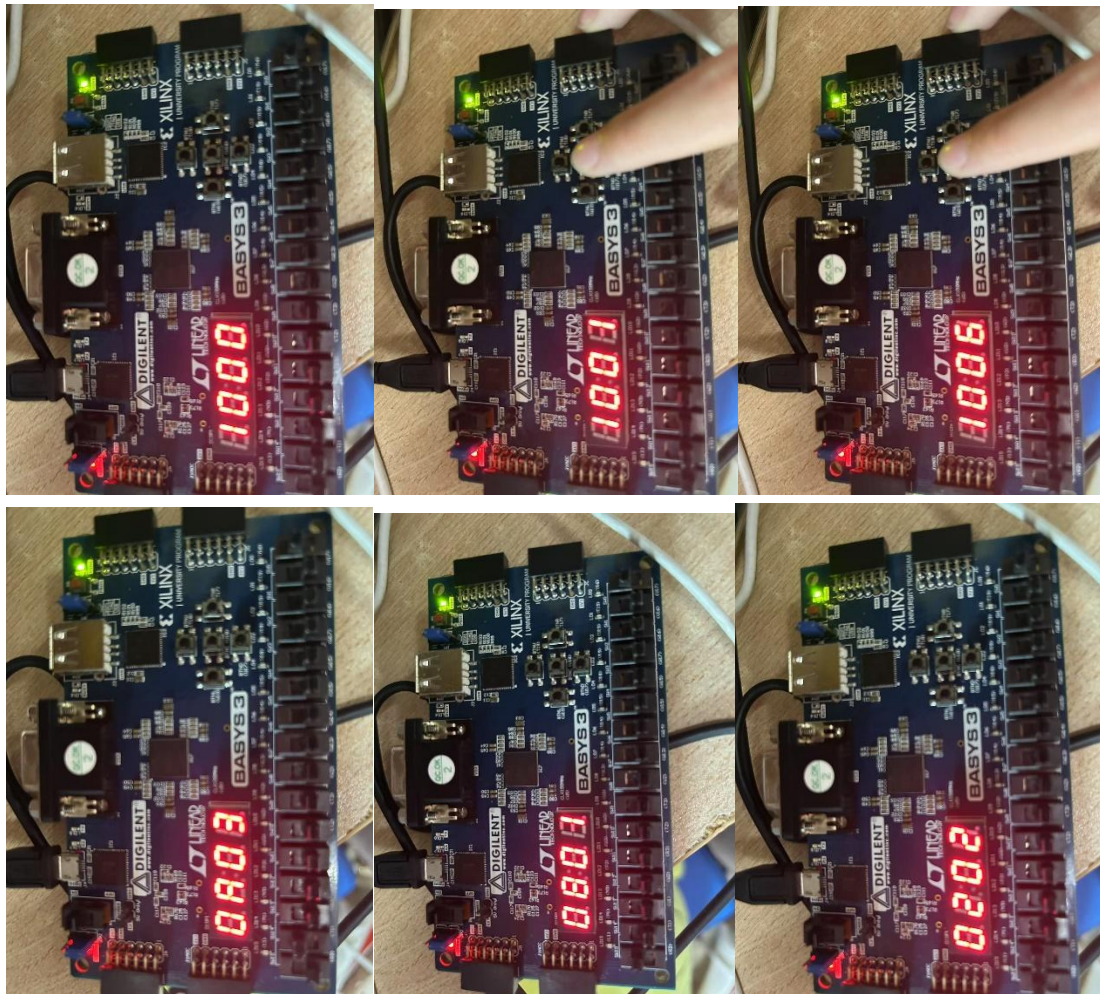




指令 4:



指令 5:



## 12. 编译器

```
#include <iostream>
#include <bitset>
#include <string>
#include <sstream>
#include <unordered_map>
```

```
std::unordered_map<std::string, std::string> opcode_map = {
    {"addiu", "001001"},
    {"ori", "001101"},
    {"xori", "001110"},
    {"sub", "000000"},
    {"and", "000000"},
    {"sll", "000000"},
    {"beq", "000100"},
    {"jal", "000011"},
    {"slt", "000000"},
```



```

        {"slti", "001010"},
        {"add", "000000"},
        {"bne", "000101"},
        {"bltz", "000001"},
        {"andi", "001100"},
        {"j", "000010"},
        {"sw", "101011"},
        {"lw", "100011"},
        {"jr", "000000"},
        {"halt", "111111"}
    };

// 函数将指令的寄存器号转换为 5 位二进制
std::string register_to_binary(int reg) {
    return std::bitset<5>(reg).to_string();
}

// 处理常见的格式: R 格式
std::string generate_r_format(std::string opcode, int rs, int rt, int
rd, int shamt, int funct) {
    std::string instruction = opcode_map[opcode] +
register_to_binary(rs) + register_to_binary(rt)
+ register_to_binary(rd) +
std::bitset<5>(shamt).to_string()
+ std::bitset<6>(funct).to_string();

    return instruction;
}

// 处理常见的格式: I 格式
std::string generate_i_format(std::string opcode, int rs, int rt, int
immediate) {
    std::string instruction = opcode_map[opcode] +
register_to_binary(rs)
+ register_to_binary(rt) +
std::bitset<16>(immediate).to_string();
    return instruction;
}

// 处理常见的格式: J 格式
std::string generate_j_format(std::string opcode, int address) {
    std::string instruction = opcode_map[opcode] +
std::bitset<26>(address).to_string();
    return instruction;
}

```

```

// 汇编指令转化为机器码
std::string assembly_to_machine_code(std::string assembly_line) {
    std::istringstream ss(assembly_line);
    std::string opcode, rs, rt, rd;
    int rs_num, rt_num, rd_num, shamt, funct, immediate, address;

    // 提取指令
    ss >> opcode;

    if (opcode == "addiu" || opcode == "ori" || opcode == "xori" ||
opcode == "beq" || opcode == "bne" ||
        opcode == "slti" || opcode == "addi" || opcode == "andi") {
        // 处理 I 格式指令
        ss >> rs >> rt >> immediate;
        rs_num = std::stoi(rs.substr(1));
        rt_num = std::stoi(rt.substr(1));
        return generate_i_format(opcode, rs_num, rt_num, immediate);
    } else if (opcode == "sub" || opcode == "and" || opcode == "sll"
|| opcode == "slt" || opcode == "add" || opcode == "bne") {
        // 处理 R 格式指令
        ss >> rs >> rt >> rd;
        rs_num = std::stoi(rs.substr(1));
        rt_num = std::stoi(rt.substr(1));
        rd_num = std::stoi(rd.substr(1));
        return generate_r_format(opcode, rs_num, rt_num, rd_num, 0,
32);
    } else if (opcode == "jal" || opcode == "j") {
        // 处理 J 格式指令
        ss >> address;
        return generate_j_format(opcode, address);
    } else if (opcode == "halt") {
        // halt 指令
        return "11111100000000000000000000000000";
    }

    return "Invalid Instruction";
}

```

## 六、实验心得

这次多周期 cpu 的设计比单周期的时间短，任务紧，我在单周

期 cpu 的基础上增加了一些模块，修改了控制信号。过程自然是有些困难的，好在自己一点点检查出来了几处接线的问题，并且在同学的帮助下完成了控制信号。通过这次实验，我不仅加深了对多周期 CPU 设计原理的理解，还提高了自己解决问题的能力，学会了如何在面对困难时保持冷静和耐心，逐步排查并解决问题。